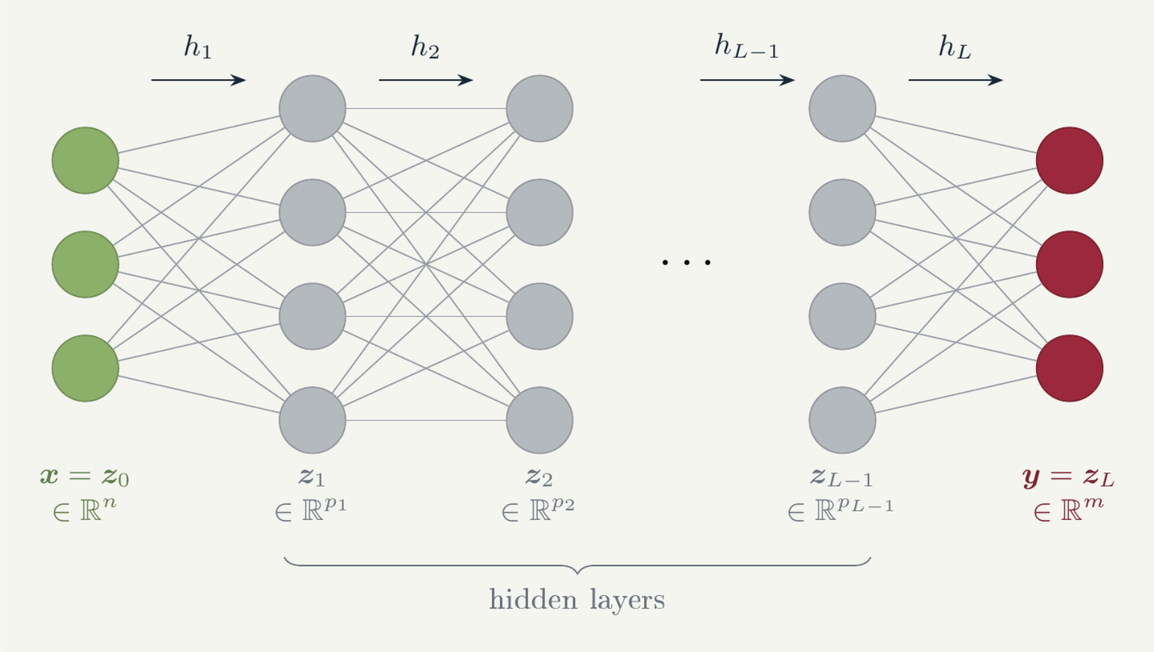
PINNS COURSE · PART 1

Introduction to Machine Learning: Supervised Learning

Neural networks, losses and optimizers

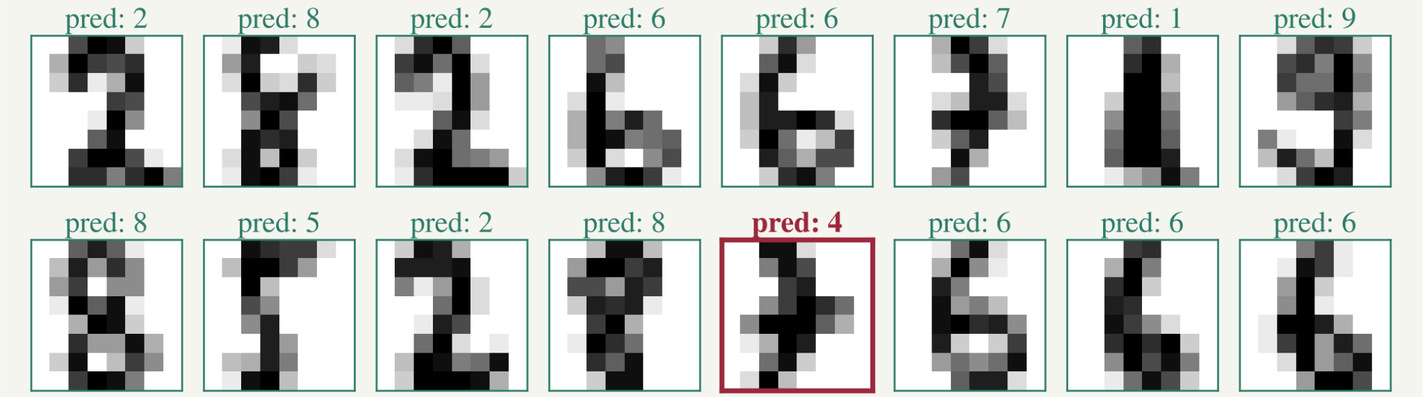
Nassim RAZAALY

v1 · 14/10/26



Machines that see: recognizing images

Supervised: learn from labelled examples



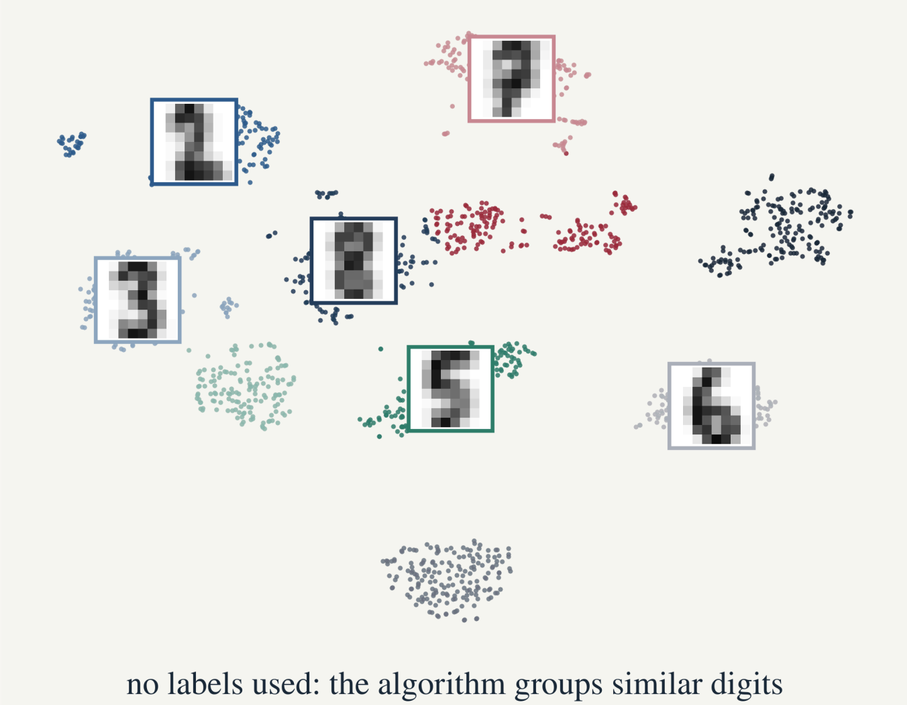
MLP, 64 hidden neurons: test accuracy = 97.1 % on 450 unseen digits

x = 8×8 image = 64 numbers → **y** = digit 0…9.

A small neural network trained on **1 347** labelled digits gets **97.1 %** right on **450** digits it has never seen (one error shown in red).

Same idea, much bigger: photos (“cat”), faces (phone unlock), medical scans (“tumour or not”).

Unsupervised: no labels, find groups



no labels used: the algorithm groups similar digits

Each dot is one image. Similar images land close together: groups appear **without telling the machine which digit is which**.

Machines that create: images and text

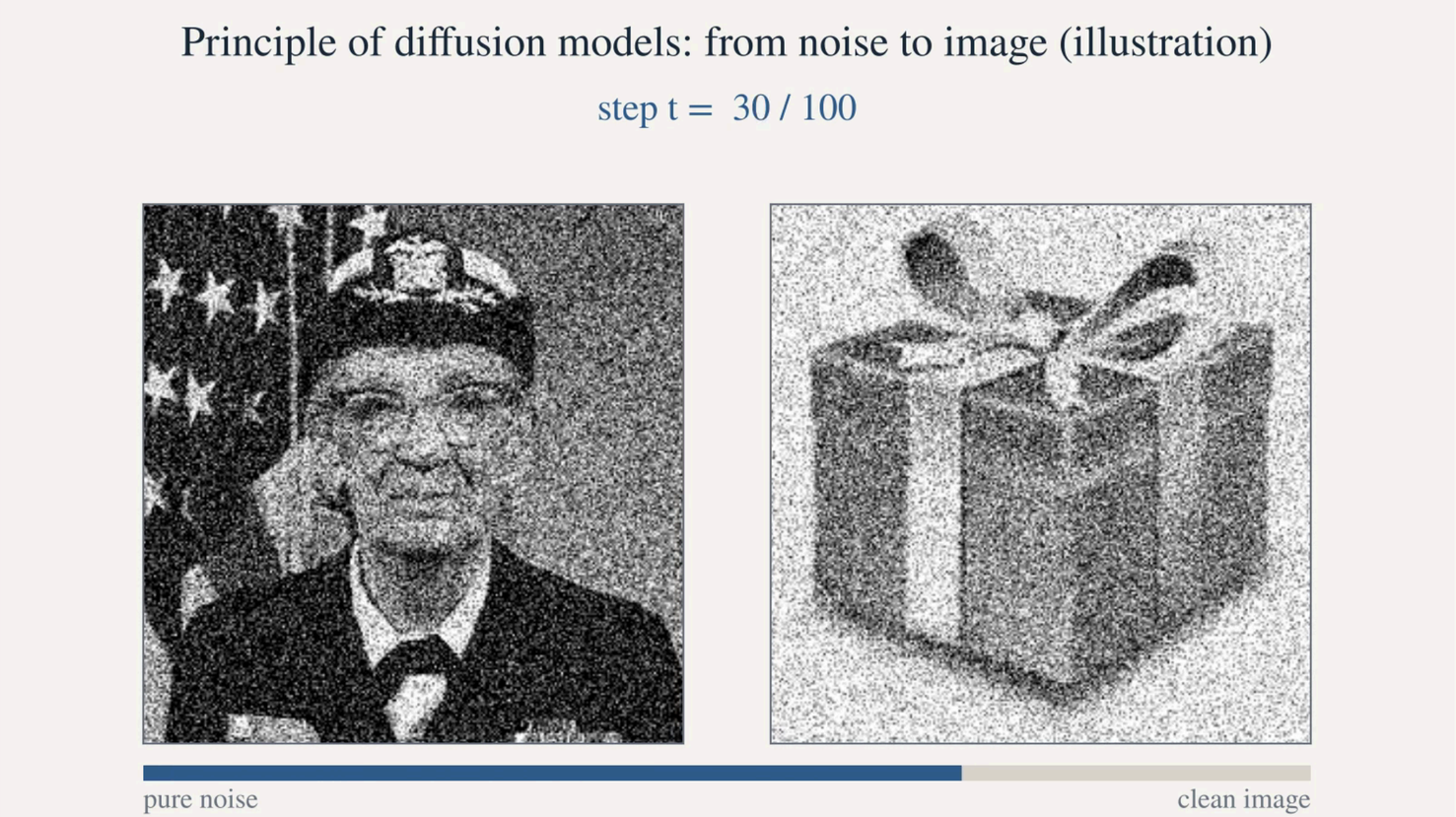


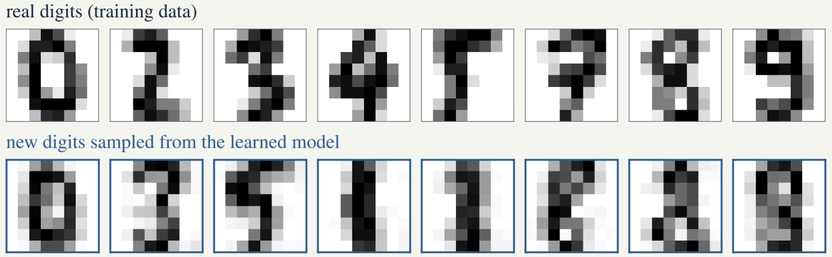
Illustration of the principle: noise added to two real pictures, played backwards. A real generator **learns** to remove the noise, and starts from noise it has never seen.

Images

Stable Diffusion, DALL·E: start from **pure noise**; a network removes a little noise at each step; after tens of steps, a **new** image.

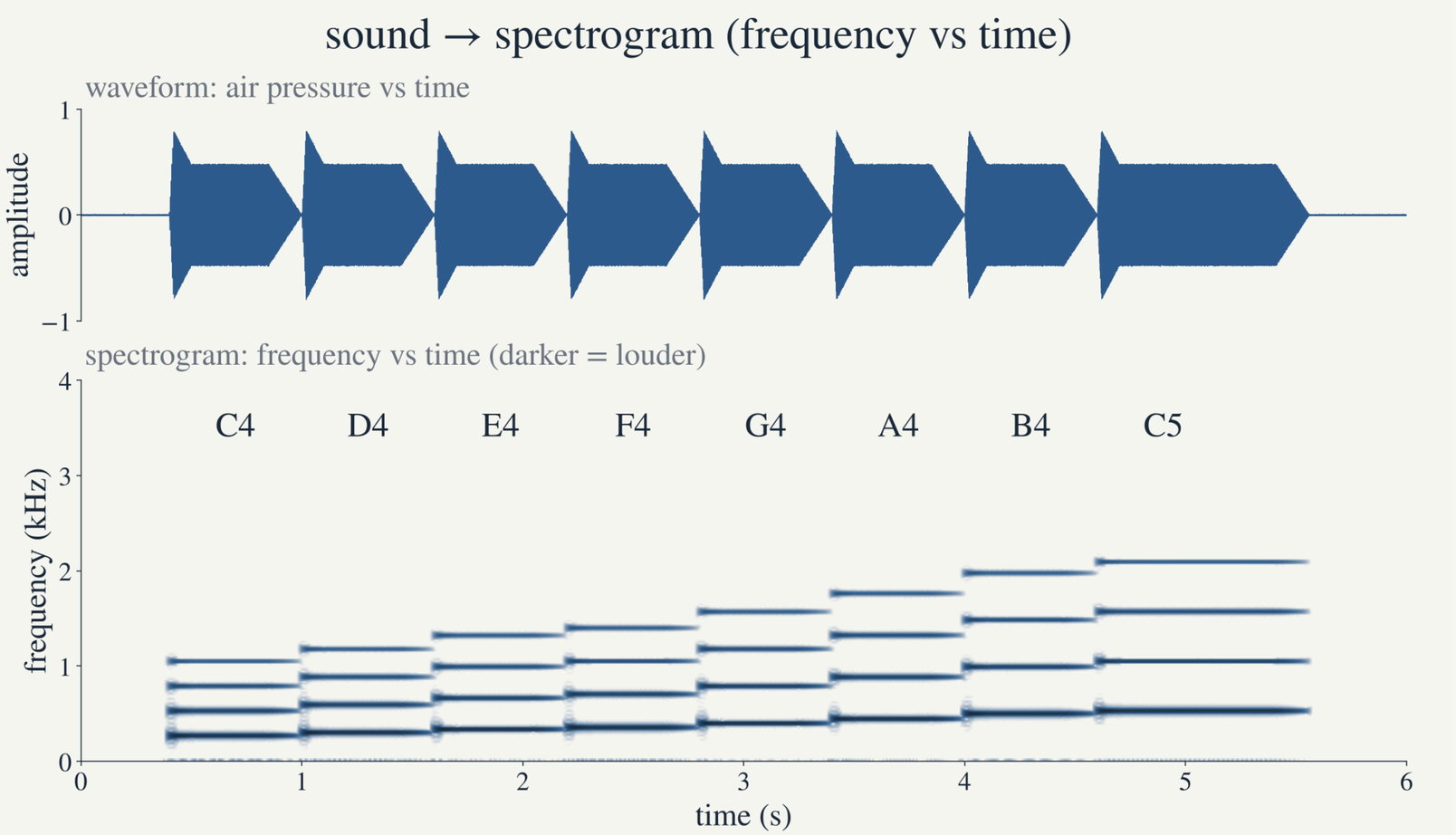
Text

ChatGPT, Claude: **x** = text so far → **y** = probability of each possible next word. Pick one, append, repeat.



Small and real: a model learns how the 1 797 digits are distributed, then draws **new** ones (bottom row).

Machines that hear: sound becomes an image



Sound = air pressure vs time: here 16 000 numbers per second.

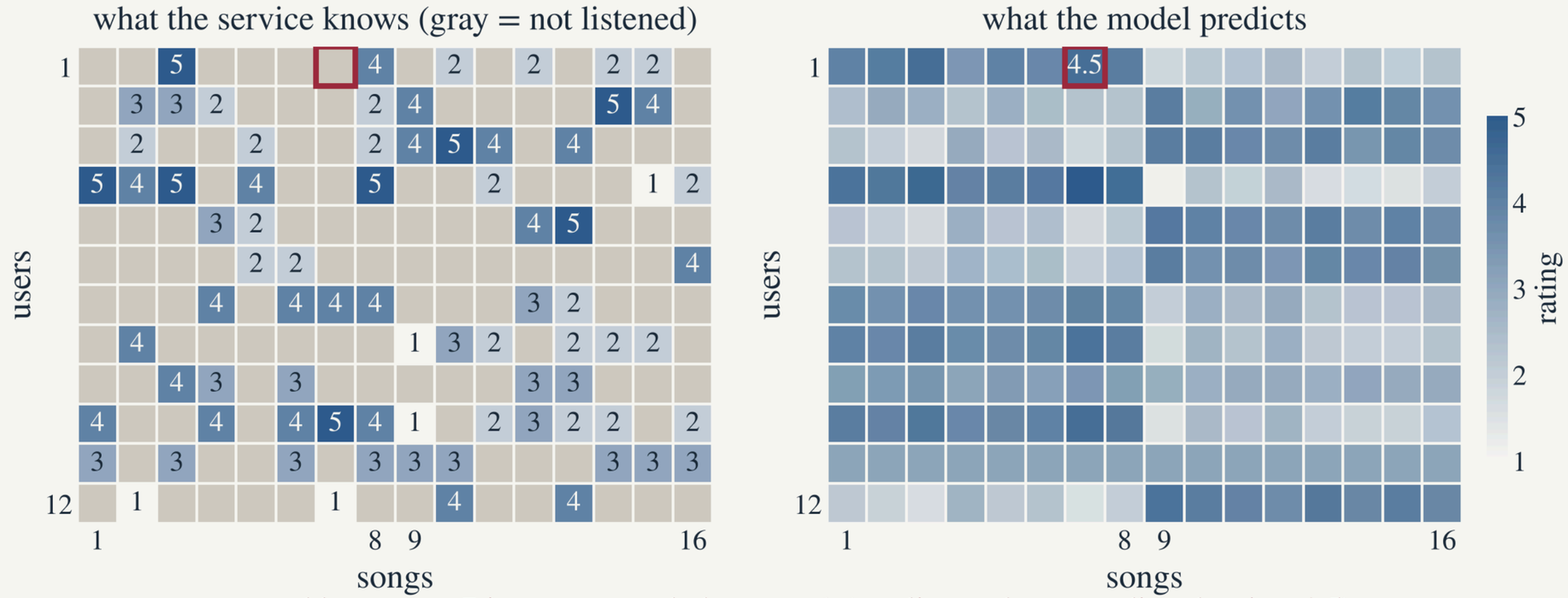
Spectrogram = which frequencies, when. Each note = a stack of lines (its harmonics). It is **an image**.

Speech → text (subtitles, assistants): a network reads the spectrogram.

Text → speech, voice conversion: a network writes a spectrogram, turned back into sound.

Synthetic scale C4 → C5 (with sound): waveform on top, spectrogram below.

Machines that recommend: filling in a table



red box: song 7 is recommended to user 1 (not listened yet, predicted rating 4.5)

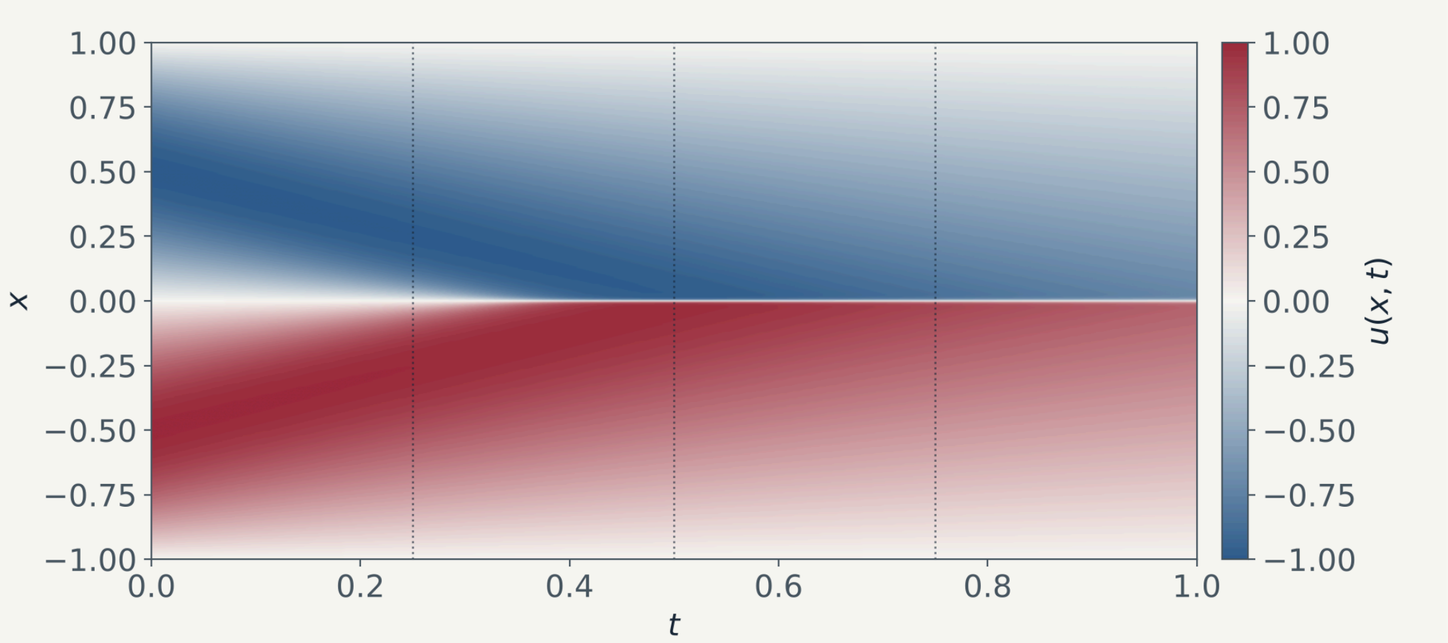
Data: 12 users × 16 songs, ratings 1–5; **60 %** unknown (grey). Toy data.

Model: each user and each song gets 2 numbers (“tastes”), learned from the known ratings.

Check on the hidden ratings: error **0.65** star, vs 1.12 when guessing the average.

At scale: Spotify, Netflix, YouTube; millions of users, plus listening history and audio.

Machines for science: from proteins to PDEs



Burgers equation $\partial u/\partial t + u\,\partial u/\partial x = (0.01/\pi)\,\partial^2 u/\partial x^2$, $u(x, 0) = -\sin(\pi x)$: the solution steepens into a sharp front at $x = 0$.

A PINN learns this $u(x, t)$ with a neural network: $\mathbf{x} = (x, t) \to \mathbf{y} = u$. This is where the course goes.

Biology

AlphaFold: amino-acid sequence → 3D shape of the protein. Over 200 million structures predicted. Nobel Prize in Chemistry 2024.

Weather

GraphCast (2023): 10-day global forecast in under a minute; more accurate than the reference ECMWF forecast on 90 % of 1 380 targets.

Engineering

Surrogate models: replace an expensive simulation (CFD, structures) by a network that answers in milliseconds.

Same pattern everywhere: learn x ↦ y from examples

Image recognition

Photo apps, face unlock, medical imaging.

x = image (pixels) → **y** = “cat”

Chatbots

ChatGPT, Claude: text generated one word at a time.

x = text so far → **y** = next word

Recommendation

Spotify, Netflix, YouTube: what to play next.

x = listening history → **y** = songs you will like

Translation, speech

Online translators, voice assistants, subtitles.

x = French sentence → **y** = English sentence

Science

AlphaFold, protein structures (Nobel Prize in Chemistry 2024).

x = amino-acid sequence → **y** = 3D shape

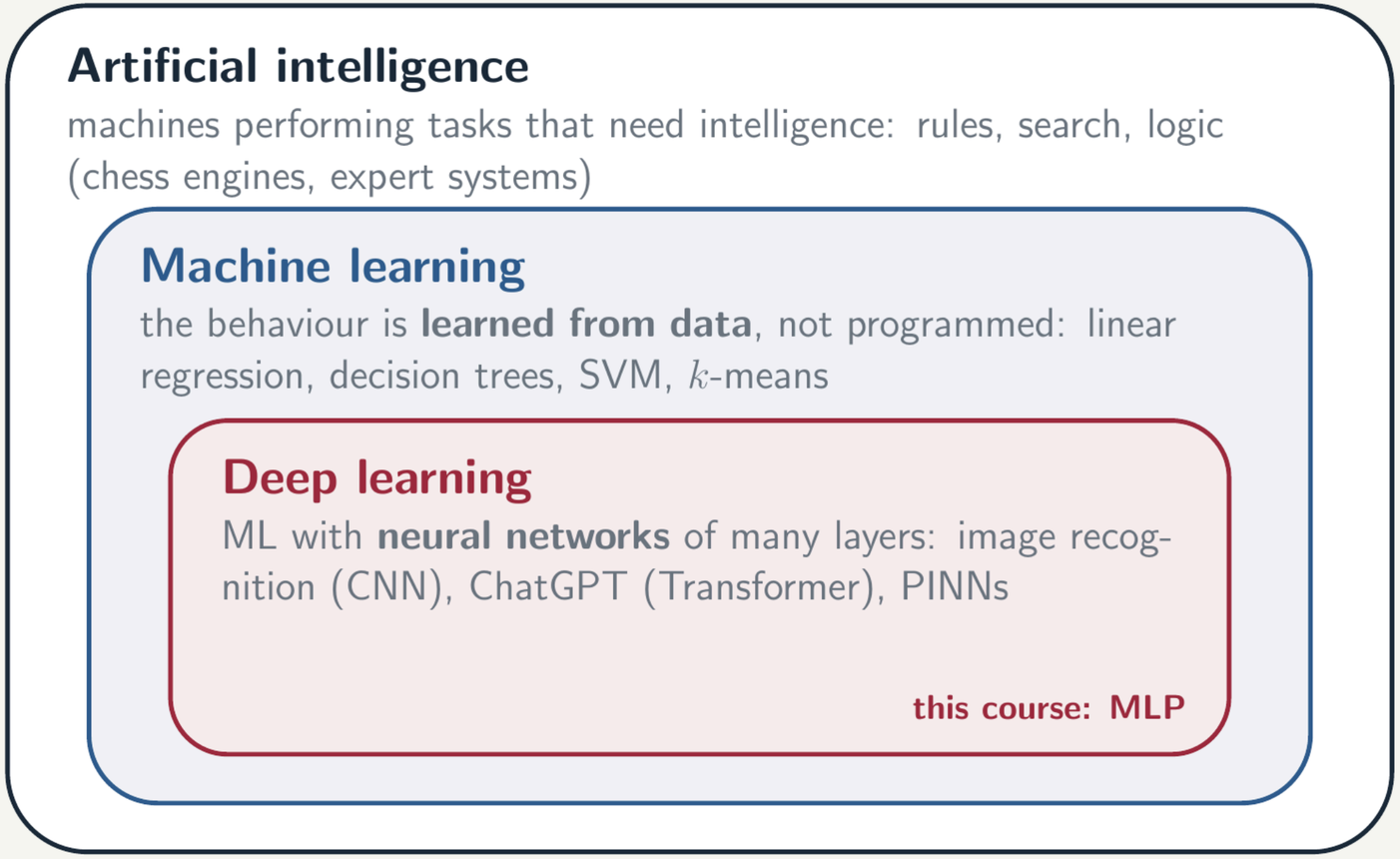
Physics, engineering

Weather forecasting, surrogate models, PINNs for PDEs.

x = (x, t) → **y** = solution u(x, t)

Nobody can write these functions by hand, but there are many examples. This course: how a neural network learns such a function.

AI, machine learning, deep learning



A definition of learning [1]

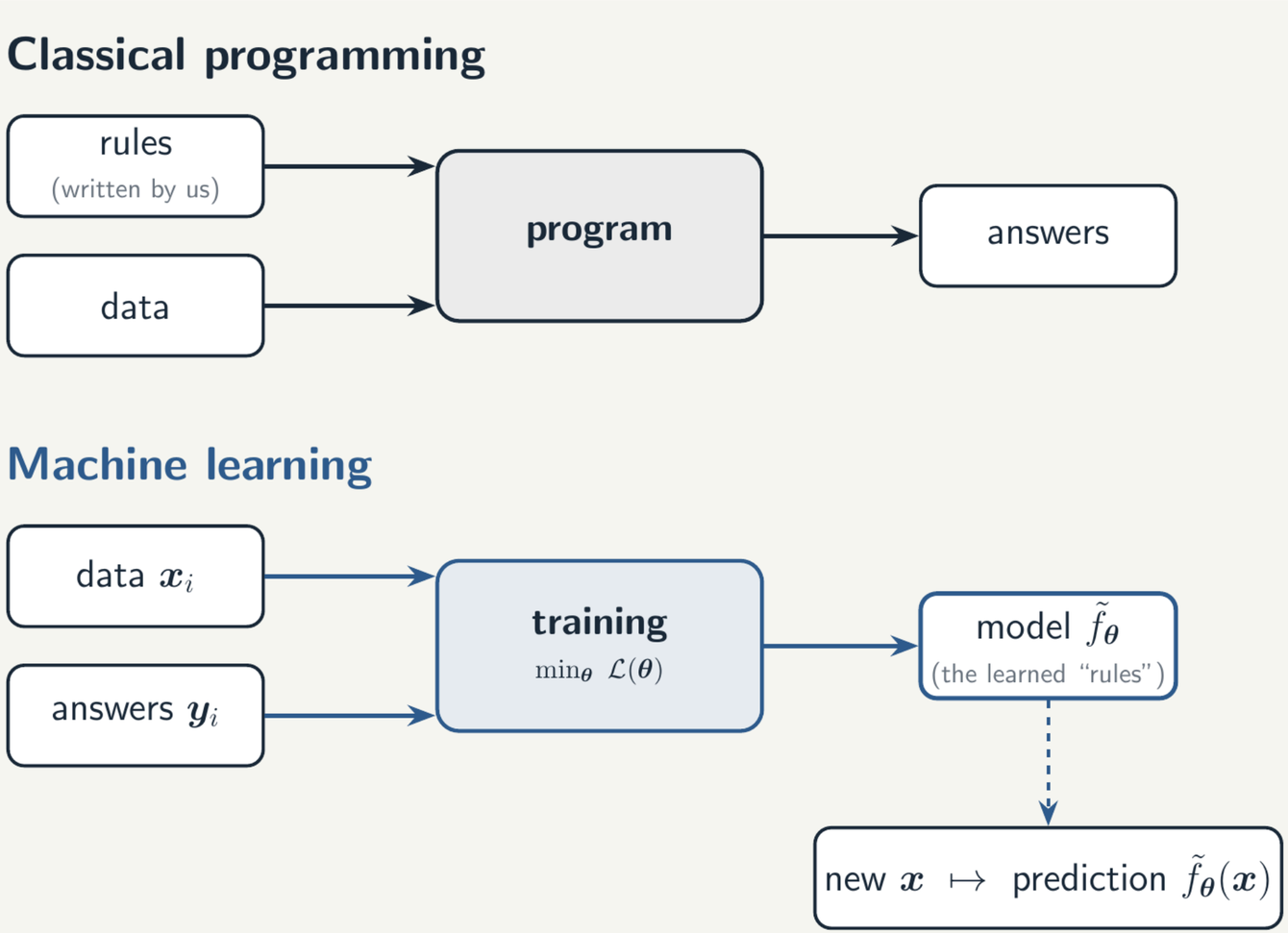
A program learns from experience E for a task T and a performance measure P if its performance on T, measured by P, improves with E.

Example: a spam filter

T sort e-mails into spam / not spam
E e-mails already marked by users
P fraction of e-mails correctly sorted

[1] T. M. Mitchell, Machine Learning, McGraw-Hill, 1997. The term “machine learning” goes back to A. Samuel (IBM), 1959.

From writing rules to learning them



When to learn?

When the rules are unknown or too complex to write (recognize a cat, translate a sentence), but **examples are plentiful**.

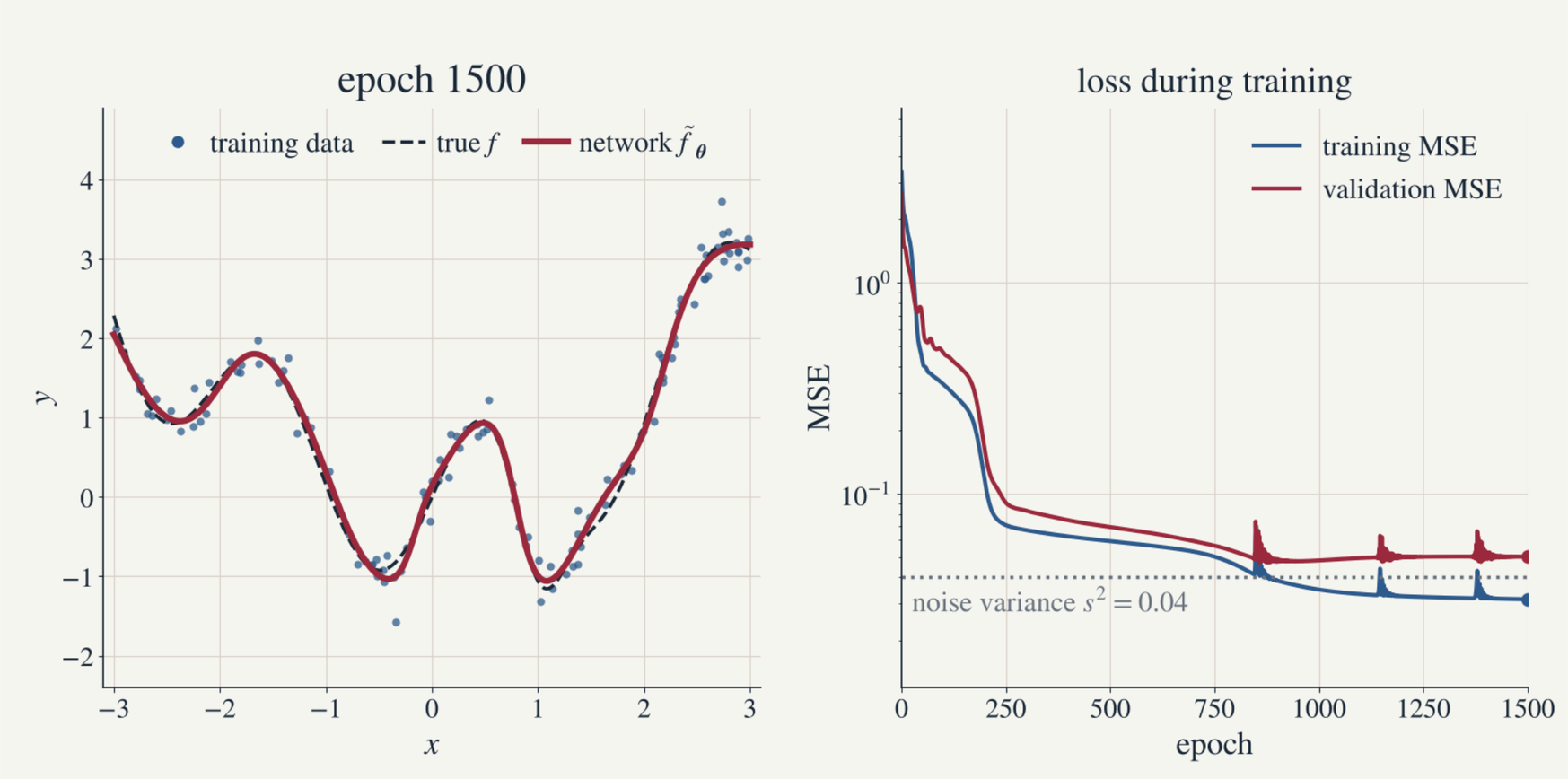
What it takes

Data, a family of models $\tilde{f}$ with parameters θ, a loss $\mathcal{L}(\theta)$ that measures errors, an optimizer that minimizes it.

The catch

A model is only as good as its data: far from the examples, its predictions are not guaranteed.

Learning = adjusting a function to examples, step by step



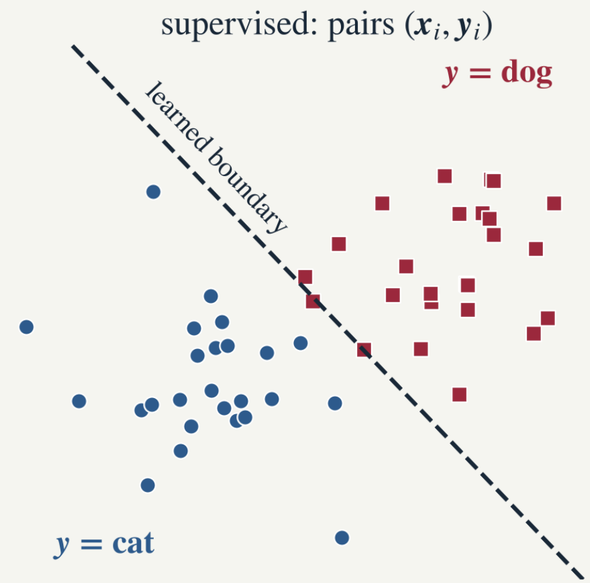
Blue dots: 120 noisy examples (x_i, y_i).

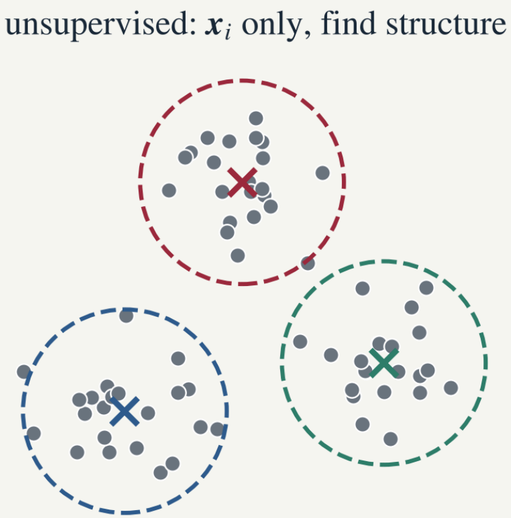
Red curve: the network's prediction. Each frame, its parameters move a little to reduce the error.

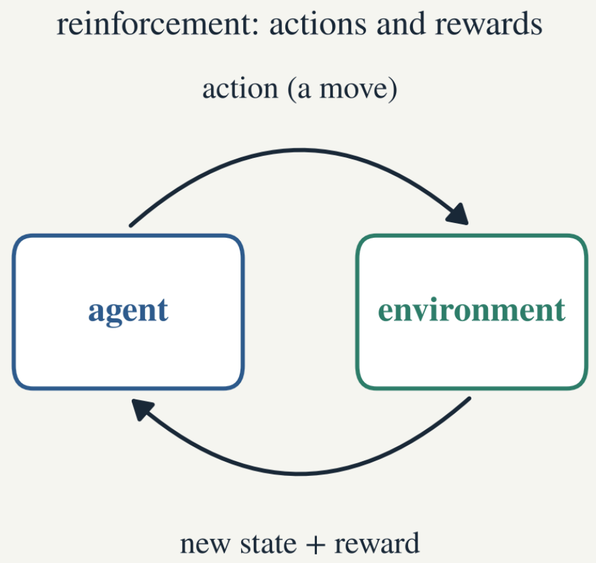
Right: the error on the training data, and on **validation** data never used for training.

Error: 3.3 → **0.03** (training), **0.05** (validation). No formula was written by hand.

Three ways to learn







Learning	Data	Goal	Examples
Supervised	pairs (x_i, y_i)	predict **y** for a new **x**	image labels, translation, weather, **PINNs**
Unsupervised	x_i only	find structure, groups	similar songs or listeners, anomaly detection
Reinforcement	actions, rewards	a strategy with high reward	AlphaGo, robotics

Real systems mix them. ChatGPT: **self-supervised** pre-training (predict the next word: the labels come from the text itself), then supervised fine-tuning and reinforcement learning from human feedback.

Seventy years in twelve dates

1950 — Turing asks “can machines think?”

1956 — Dartmouth workshop names “artificial intelligence”

1958 — Rosenblatt’s perceptron: a neuron that learns

1986 — Backpropagation trains multilayer networks

1989 — Universal approximation theorem

1998 — LeNet reads handwritten digits (cheques)

2012 — AlexNet wins ImageNet: deep learning takes off

2016 — AlphaGo beats the Go champion Lee Sedol

2017 — Transformer architecture (language models)

2019 — Physics-informed neural networks (PINNs)

2020 — AlphaFold 2: protein structures at near-experimental accuracy

2022 — ChatGPT opens to the public

WHY NOW?

Data (the web, sensors) · **compute** (GPUs) · **algorithms** (backpropagation, Adam, Transformers) · **software** (automatic differentiation: PyTorch, JAX)

NOT A STRAIGHT LINE

Two “AI winters” (1970s, around 1990) when funding and interest collapsed.

This course: neural networks for supervised learning

I · PROBLEM
data $(\mathbf{x}_i, \mathbf{y}_i)$

→

II · MODEL
MLP $\tilde{f}$ with parameters $\boldsymbol{\theta}$

→

III · TRAINING
loss $\mathcal{L}(\boldsymbol{\theta})$, data split

→

IV · OPTIMIZATION
gradients, Adam, L-BFGS

→

V · IN PRACTICE
PyTorch, a recipe

Why neural networks?

- they can approximate any continuous function (part II);
- their parameters are found by gradient methods, at any scale (part IV);
- they are differentiable with respect to their inputs: derivatives $\partial\tilde{u}/\partial x$, $\partial\tilde{u}/\partial t$ are computed exactly by automatic differentiation, which is what PINNs need.

Why the MLP?

The simplest network: layers of neurons, fully connected. It is the standard model of PINNs, and the building block of larger architectures.

Notation: vectors in bold ($\mathbf{x}$, $\mathbf{y}$, $\boldsymbol{\theta}$), N samples, d parameters, L layers.

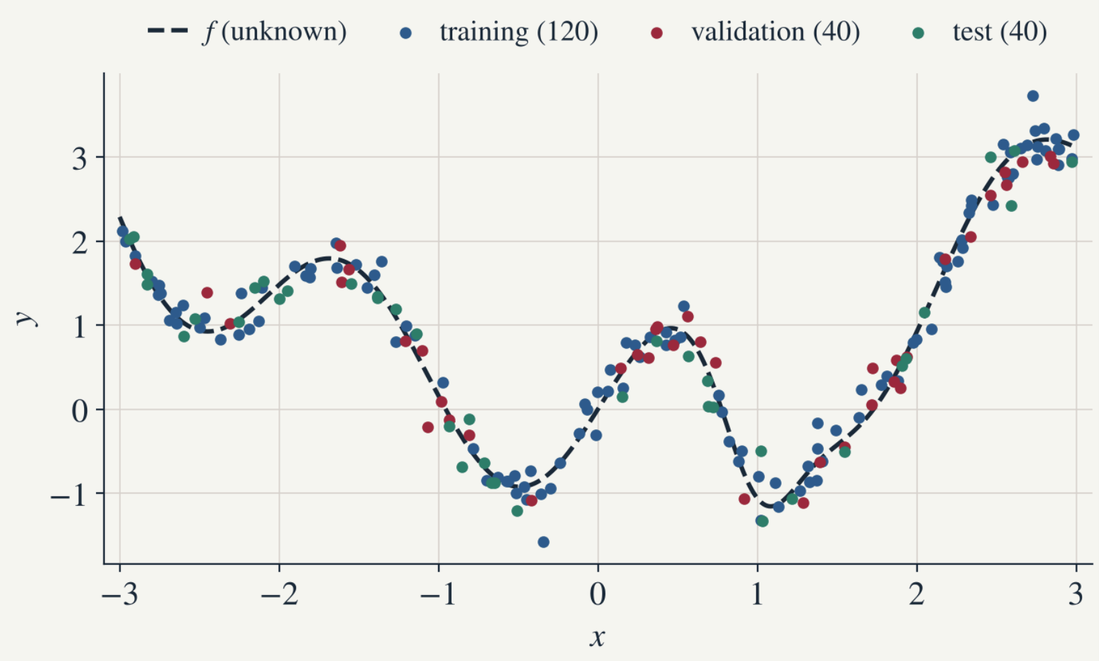
I. Learning a function from samples

$$f : \mathbb{R}^n \to \mathbb{R}^m, \qquad \boldsymbol{x} \mapsto \boldsymbol{y} = f(\boldsymbol{x}) \quad \text{unknown}$$

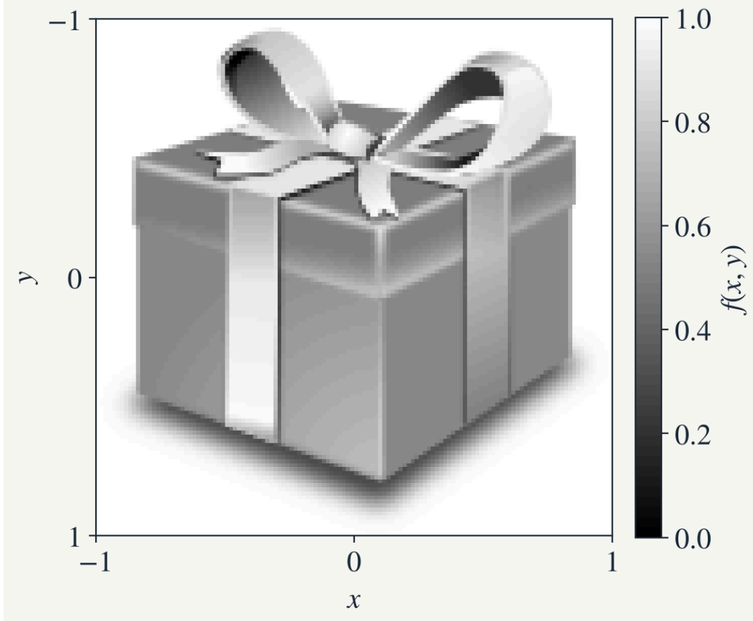
$$\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N}, \qquad \boldsymbol{y}_i = f(\boldsymbol{x}_i) + \boldsymbol{\varepsilon}_i \quad \text{(noise)}$$

find $\boldsymbol{\theta} \in \mathbb{R}^d$ such that $\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) \approx f(\boldsymbol{x})$ also for new inputs $\boldsymbol{x}$

Example 1 · n = m = 1, N = 200 noisy samples



Example 2 · an image is a function



f : [−1, 1]² → [0, 1], n = 2, m = 1, N = 128² = 16 384 pixels

Parametric model: f̃ is chosen in a family indexed by θ ∈ ℝᵈ. Learning = choosing d numbers. Here: regression (**y** continuous).

I. A first model: linear least squares

$$\tilde{f}_{\boldsymbol{\theta}}(x) = \theta_1 + \theta_2\, x, \qquad \boldsymbol{\theta} = (\theta_1, \theta_2) \in \mathbb{R}^2$$

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}\in\mathbb{R}^2} \frac{1}{N}\sum_{i=1}^{N} \big(\theta_1 + \theta_2 x_i - y_i\big)^2 = \big(X^\top X\big)^{-1} X^\top \boldsymbol{y}, \qquad X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_N \end{pmatrix}$$

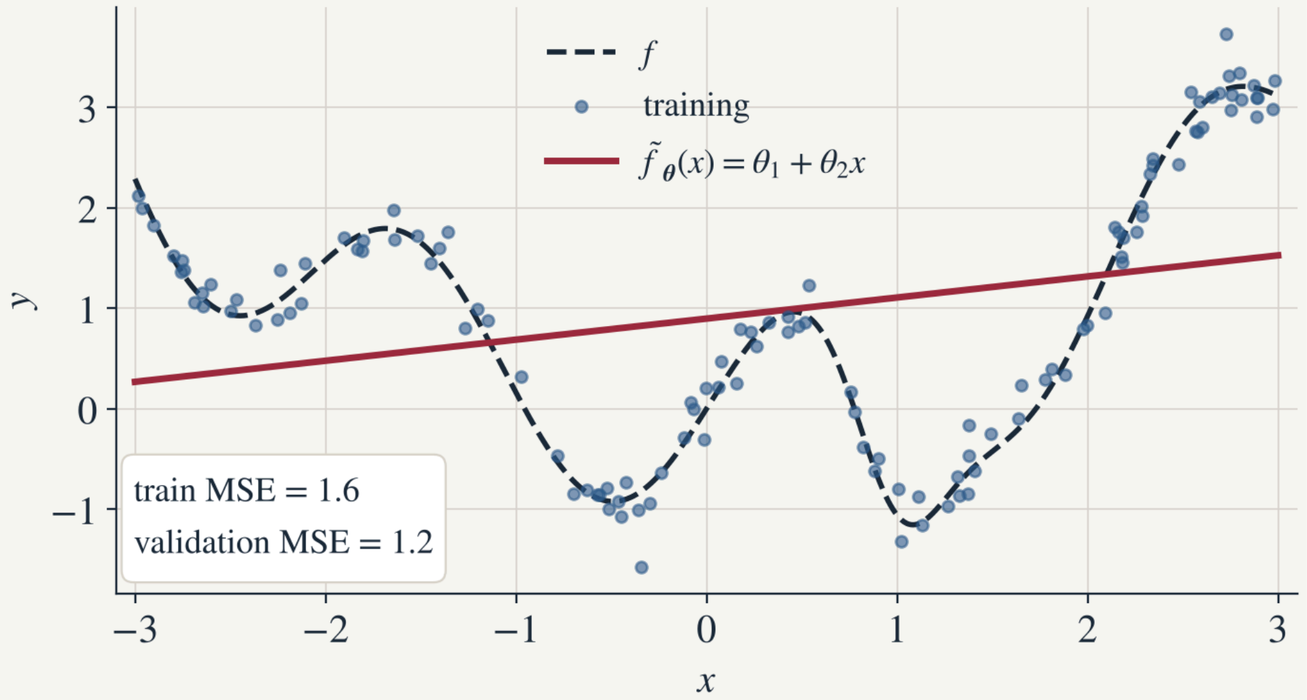
Linear in θ → convex quadratic loss → closed-form minimizer ($X^\top X$ invertible: at least 2 distinct x_i).

The three ingredients of the course

1 · **Model** $\tilde{f}$ with parameters θ → part II (MLP)

2 · **Loss** measuring the residuals → part III

3 · **Optimizer** → part IV (no closed form for an MLP)



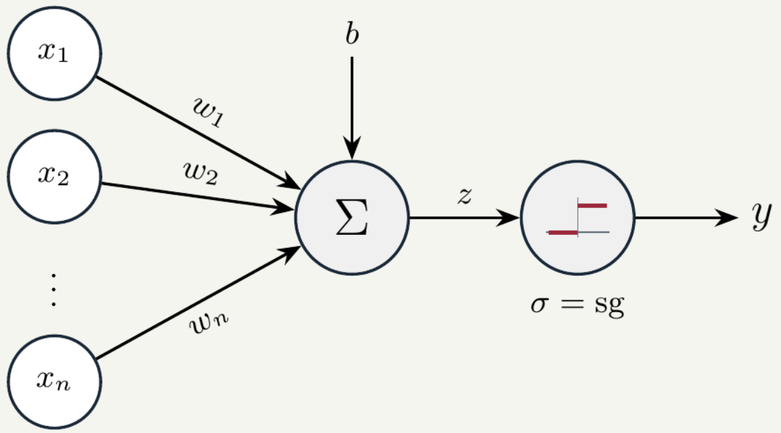
Example 1, 120 training points: θ* = (0.90, 0.21), training MSE = 1.64, while the noise variance is $s^2 = 0.04$.

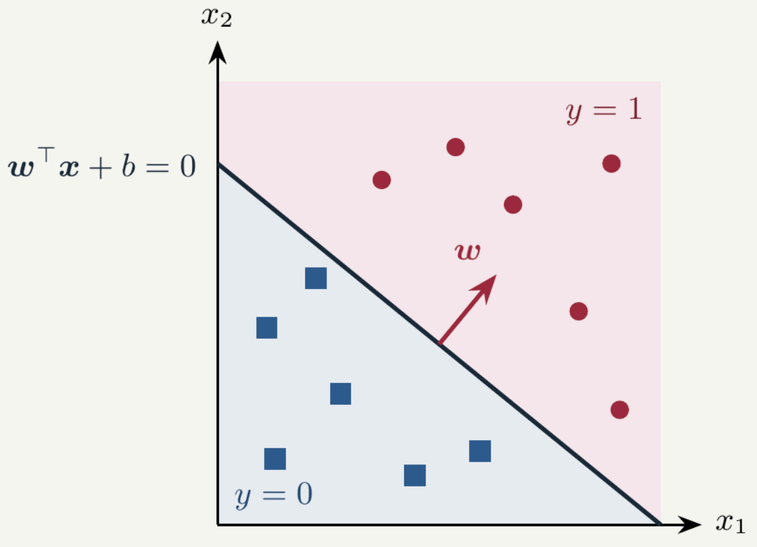
Underfitting: the family is too poor. We need a nonlinear, richer model.

II. From the perceptron to the neural unit

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{j=1}^{n} w_j x_j + b, \qquad y = \mathrm{sg}(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

Rosenblatt, 1958 [2]: weighted sum, then a threshold (step function sg). A model of a biological neuron.





Geometry: a linear decision boundary, the hyperplane $\mathbf{w}^\top\mathbf{x} + b = 0$.

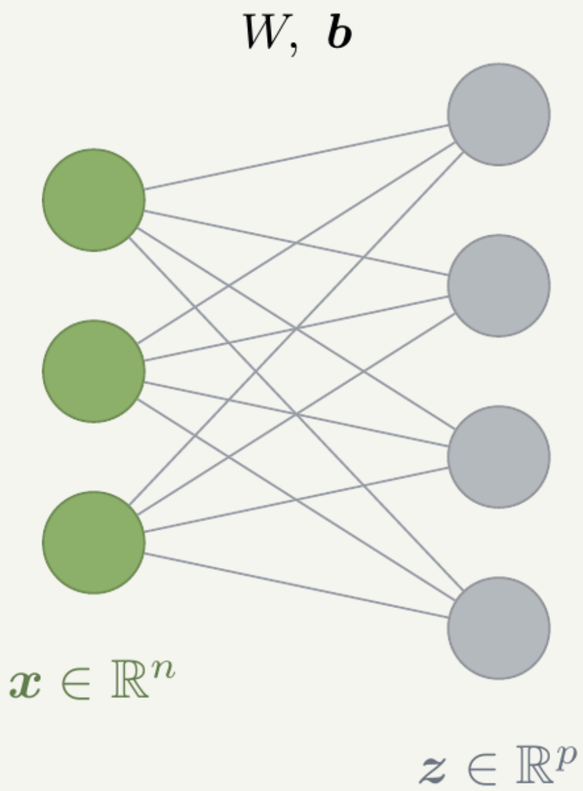
Limits: linear boundary only (no XOR); sg′ = 0 a.e.: not trainable by gradient descent.

Neural unit: sg → σ nonlinear, differentiable

$$y = \sigma\left(\boldsymbol{w}^\top \boldsymbol{x} + b\right)$$

[2] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, Psychological Review, 1958.

II. One layer: p neural units in parallel



1 layer: p(n + 1) parameters

Neuron j has its own weights $\mathbf{w}_j \in \mathbb{R}^n$ and bias b_j:

$$[\boldsymbol{z}]_j = \sigma\big(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j\big), \qquad j = 1, \dots, p$$

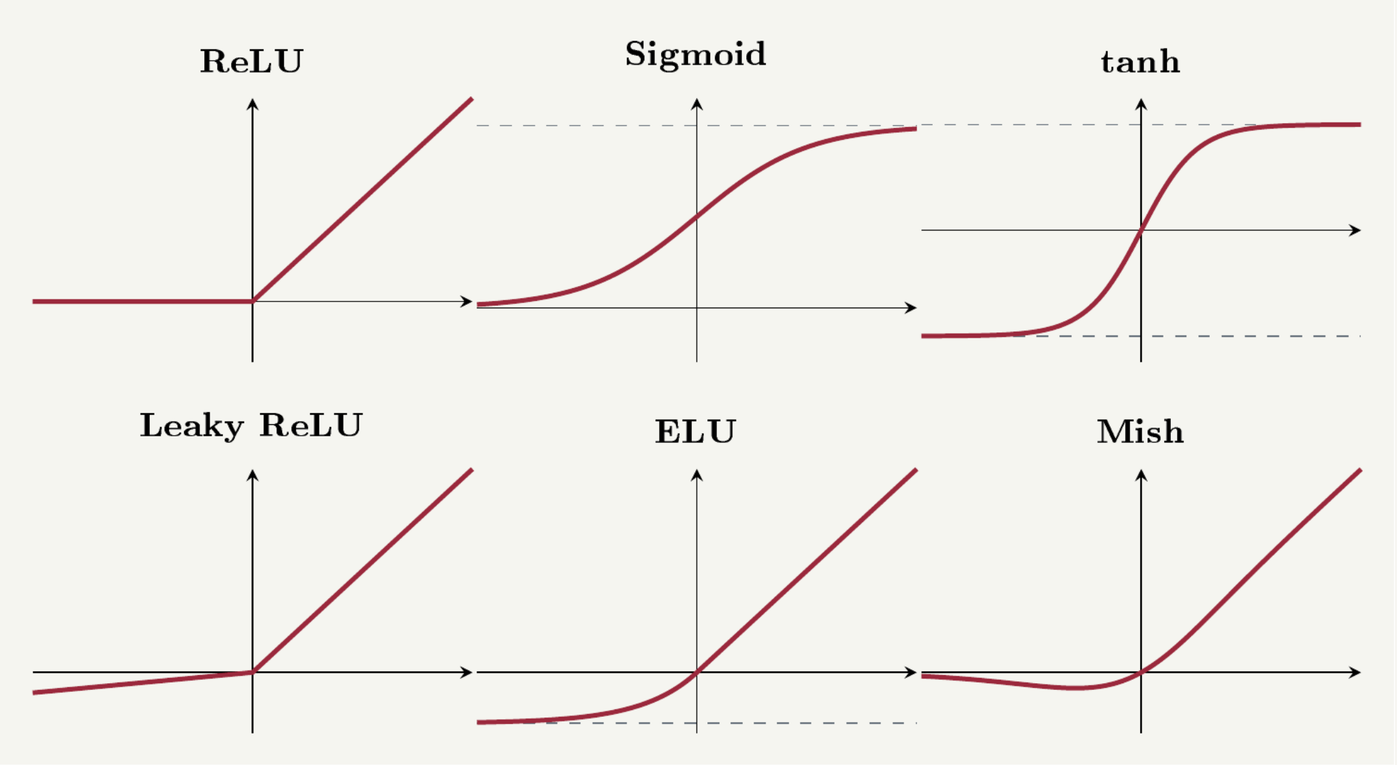
Stacking the rows $\mathbf{w}_j^\top$ in a matrix, σ applied component-wise:

$$\boldsymbol{z} = h(\boldsymbol{x}) = \sigma\big(W\boldsymbol{x} + \boldsymbol{b}\big) \in \mathbb{R}^p, \qquad W = \begin{pmatrix} \boldsymbol{w}_1^\top \\ \vdots \\ \boldsymbol{w}_p^\top \end{pmatrix} \in \mathbb{R}^{p \times n}, \quad \boldsymbol{b} \in \mathbb{R}^p$$

In PyTorch, **nn.Linear(n, p)** stores W (p, n) and **b** (p,). On a batch X of N inputs (one row per sample) it computes:

$$Z = \sigma\big(XW^\top + \mathbf{1}\,\boldsymbol{b}^\top\big) \in \mathbb{R}^{N \times p}, \qquad X = \begin{pmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times n}$$

II. Activation functions σ



σ	σ(z)	range
ReLU	max(0, z)	[0, +∞[
sigmoid	1 / (1 + e⁻ᶻ)	]0, 1[
tanh	(eᶻ − e⁻ᶻ) / (eᶻ + e⁻ᶻ)	]−1, 1[
Leaky ReLU	max(αz, z), α ≈ 10⁻²	ℝ
ELU	z, or eᶻ − 1 if z < 0	]−1, +∞[
Mish	z tanh(ln(1 + eᶻ))	[≈ −0.31, +∞[

Leaky ReLU [3], ELU [4], Mish [5]

Why nonlinear?

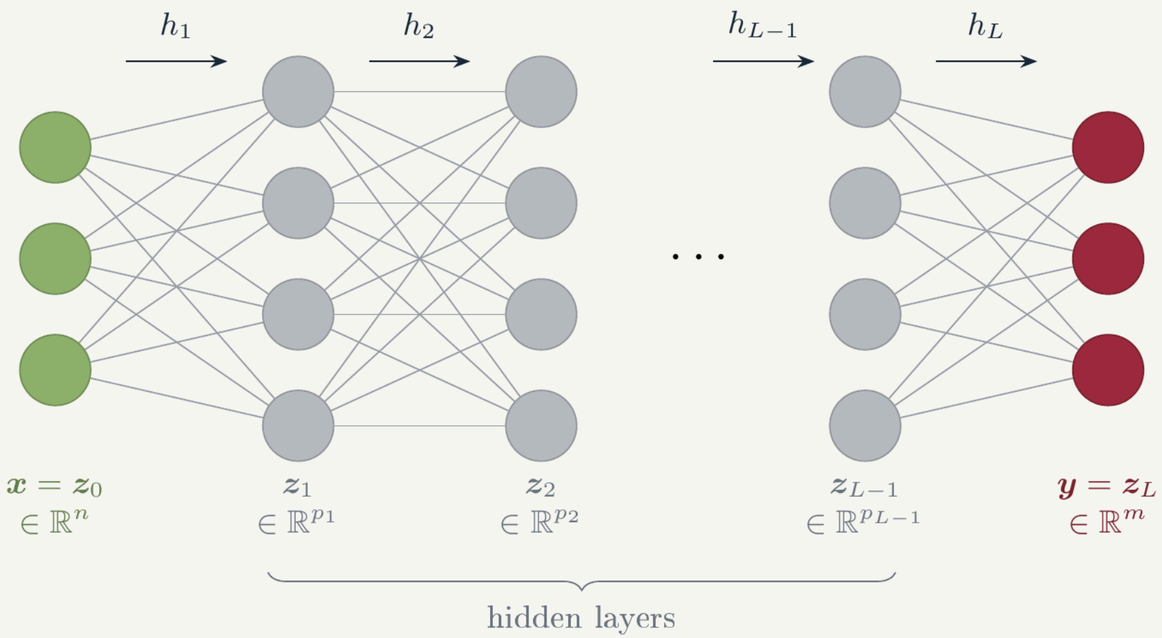
$$\sigma = \mathrm{id}: \quad W_2\big(W_1\boldsymbol{x} + \boldsymbol{b}_1\big) + \boldsymbol{b}_2 = \underbrace{W_2 W_1}_{W}\,\boldsymbol{x} + \underbrace{W_2\boldsymbol{b}_1 + \boldsymbol{b}_2}_{\boldsymbol{b}} \quad \text{affine, whatever the depth}$$

Saturation

sigmoid, tanh: σ′(z) → 0 when |z| → ∞, so gradients become small.

[3] Maas et al., ICML Workshop 2013. [4] Clevert et al., ICLR 2016. [5] Misra, BMVC 2020.

II. The multilayer perceptron (MLP): a composition of layers



$$\boldsymbol{z}_0 = \boldsymbol{x}, \qquad \boldsymbol{z}_\ell = h_\ell(\boldsymbol{z}_{\ell-1}), \quad \ell = 1, \dots, L, \qquad \boldsymbol{y} = \boldsymbol{z}_L$$

Depth L, **widths** p_1, …, p_{L-1}: layers 1 to $L-1$ are *hidden*.

Hyperparameters (chosen, not learned): L, the widths, the activations.

Last layer for regression: no activation, $\sigma_L = \mathrm{id}$.

Names: MLP = fully connected feedforward network, “vanilla” network.

$$h_\ell(\boldsymbol{z}) = \sigma_\ell\big(W_\ell\, \boldsymbol{z} + \boldsymbol{b}_\ell\big), \qquad W_\ell \in \mathbb{R}^{p_\ell \times p_{\ell-1}}, \quad \boldsymbol{b}_\ell \in \mathbb{R}^{p_\ell}, \qquad p_0 = n, \quad p_L = m$$

$$\boldsymbol{y} = \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) = \big(h_L \circ h_{L-1} \circ \cdots \circ h_1\big)(\boldsymbol{x}), \qquad \boldsymbol{\theta} = \big\{W_\ell,\ \boldsymbol{b}_\ell\big\}_{\ell=1}^{L}$$

II. How many parameters? d = dim θ

$$\text{layer } \ell: \quad \underbrace{p_\ell\, p_{\ell-1}}_{\text{weights}} + \underbrace{p_\ell}_{\text{biases}} \quad \Rightarrow \quad d = \sum_{\ell=1}^{L} p_\ell \left(p_{\ell-1} + 1\right)$$

Example: a PINN for u(x, t)

$(p_0, \ldots, p_4) = (2, 20, 20, 20, 1)$

$d = 20\cdot 3 + 20\cdot 21 + 20\cdot 21 + 1\cdot 21$

$= \mathbf{921}$

Orders of magnitude

$\approx 10^3$

a PINN (the example above)

$2.6 \cdot 10^7$

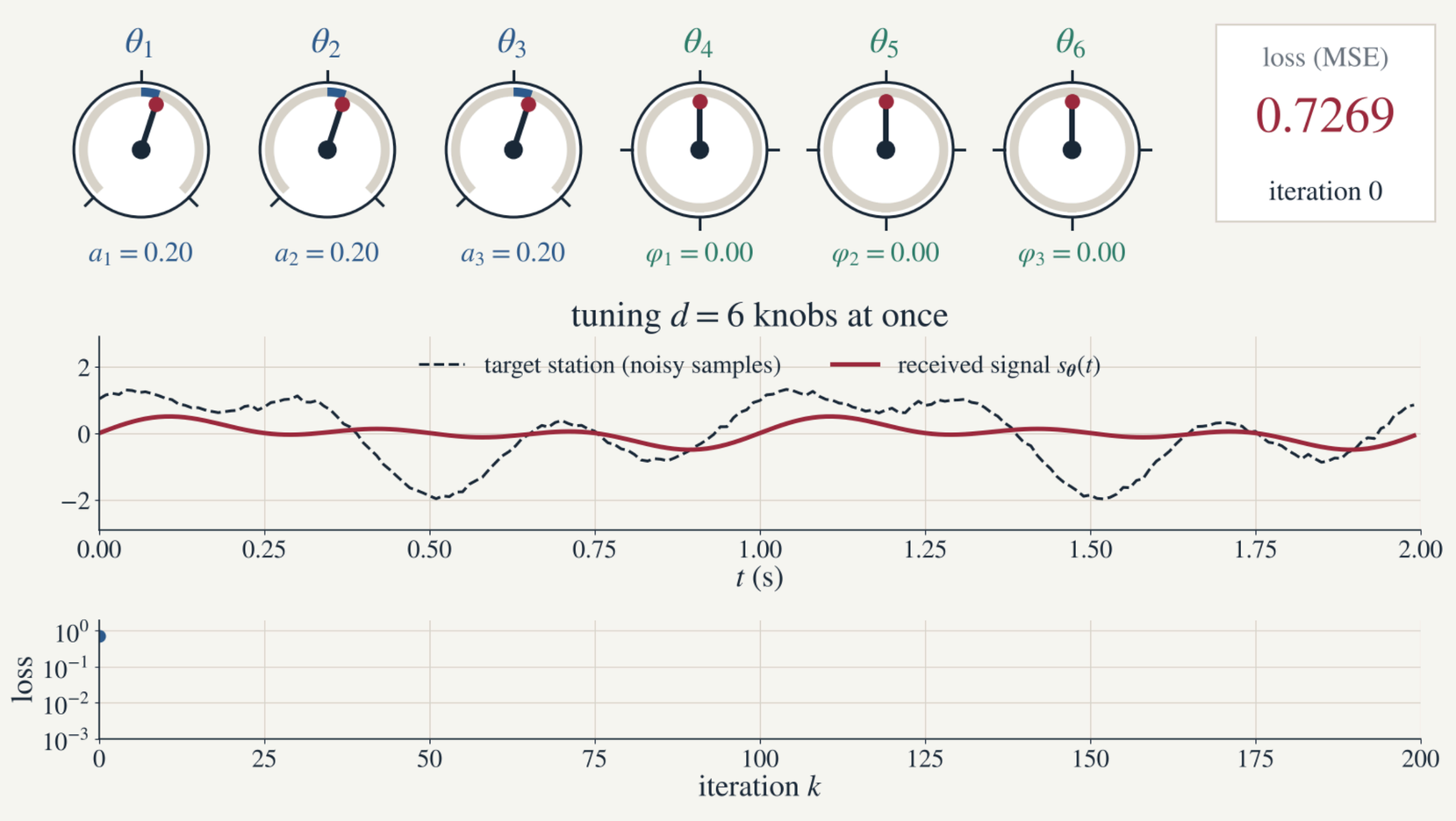
ResNet-50, image recognition (2015)

$1.75 \cdot 10^{11}$

GPT-3, language model (2020)

Training = choosing these d numbers together: a **nonlinear optimization problem in high dimension**.

II. Analogy: tuning a radio with d knobs



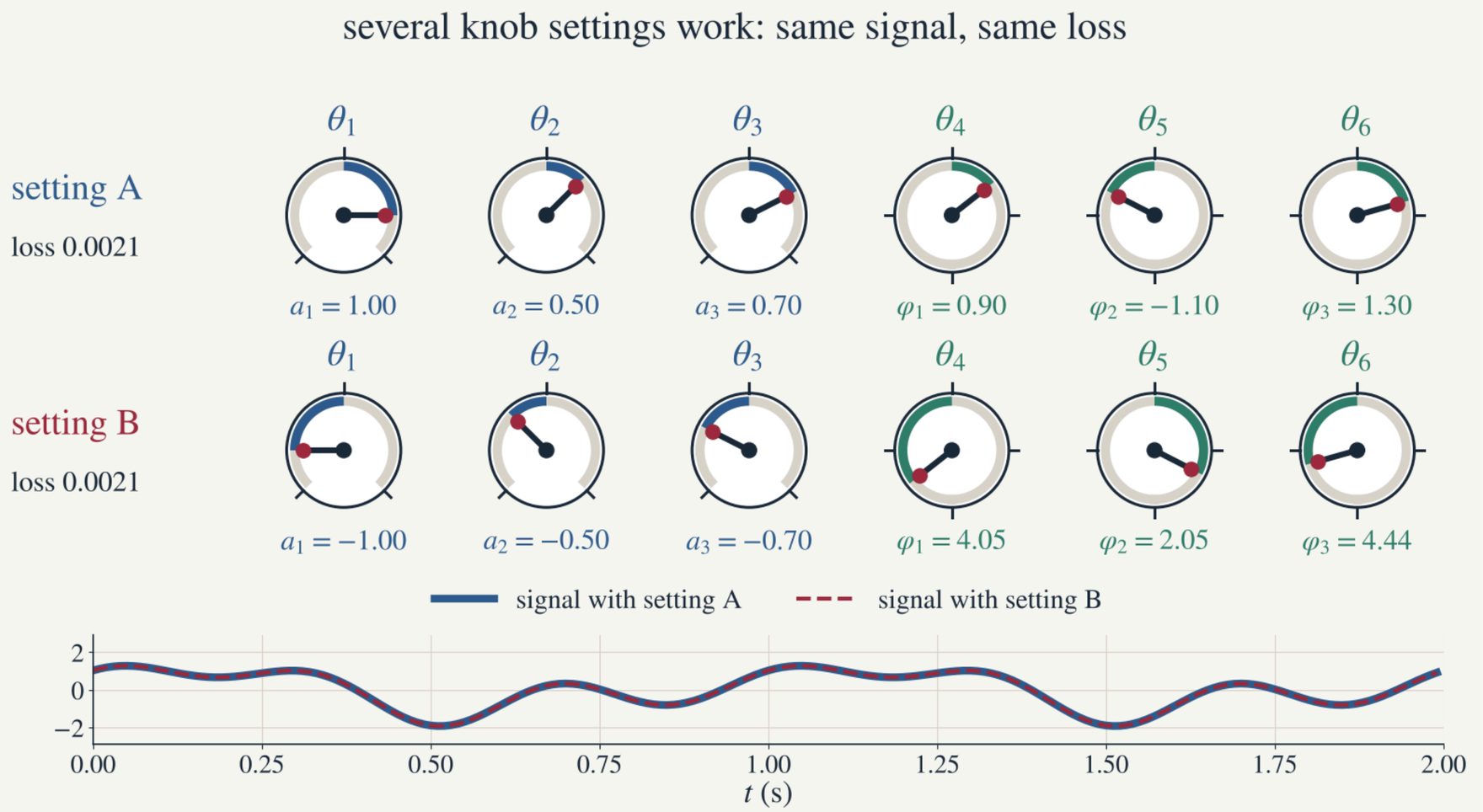
6 knobs θ = (a_1, a_2, a_3, φ_1, φ_2, φ_3):
$s(t) = \Sigma\ a_j \sin(2\pi f_j t + \varphi_j)$

Goal: match the station = make the error L(**θ**) small.

Knobs **interact**: turn them all **together**, by **small moves** that lower the error.

More knobs, harder: a network has $d = 10^3 \ldots 10^{11}$ of them.

II. Several knob settings give the same sound



Start from another θ_0 → other knob values, **same signal, same error**.

Here: flip the sign of an amplitude and turn its phase by half a turn (π).

Neural networks too: **many θ** are equally good. We need **one** of them, not "the" solution.

II. Why MLPs? The universal approximation theorem [6–8]

Theorem (universal approximation). Let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous and *not a polynomial*, and $K \subset \mathbb{R}^n$ compact. For every continuous $f : K \to \mathbb{R}^m$ and every $\varepsilon > 0$, there exist a width p and $W \in \mathbb{R}^{p \times n}$, $\boldsymbol{b} \in \mathbb{R}^p$, $V \in \mathbb{R}^{m \times p}$ such that

$$\sup_{\boldsymbol{x} \in K} \left\| f(\boldsymbol{x}) - V\, \sigma(W\boldsymbol{x} + \boldsymbol{b}) \right\| < \varepsilon.$$

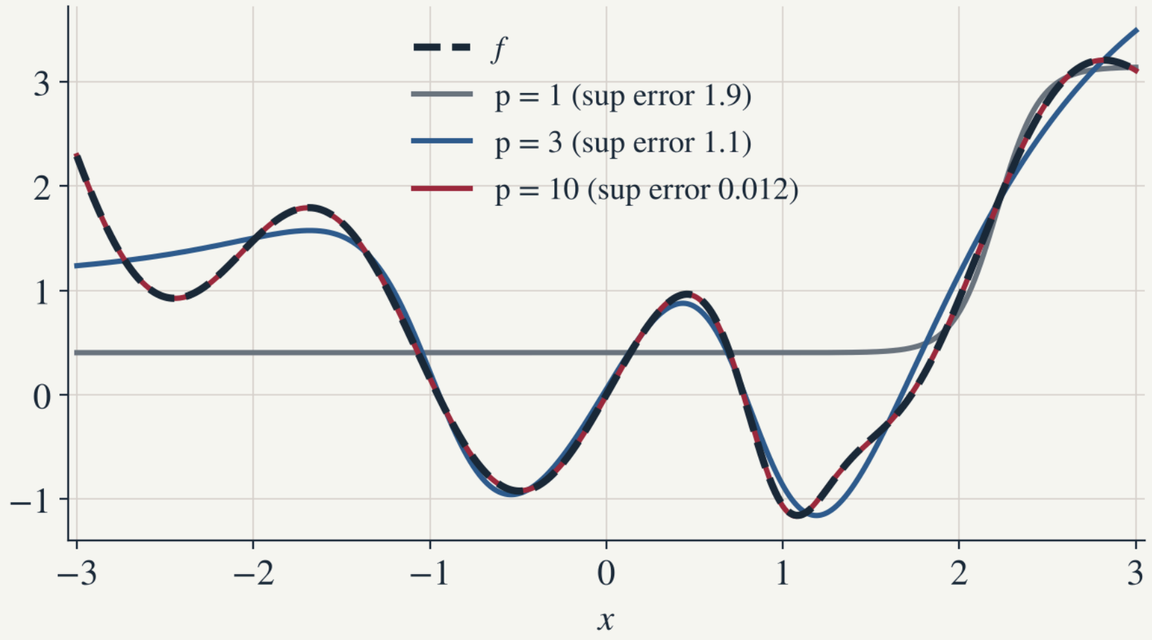


Illustration: one hidden layer of p tanh neurons fitted to f. With p = 10 the network matches f.

Nonlinearity is essential: σ = id is a polynomial, excluded.

Existence only: says nothing about the width p needed, how to find θ, or generalization.

Depth: some functions need exponentially many neurons with one hidden layer, few with more layers (Telgarsky; Eldan & Shamir, COLT 2016).

[6] Cybenko, Math. Control Signals Syst., 1989. [7] Hornik, Neural Networks, 1991. [8] Leshno, Lin, Pinkus, Schocken, Neural Networks, 1993.

III. The loss function: what we minimize

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}), \qquad \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} J\big(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i\big)$$

Loss = cost function = *empirical risk*: an average of a pointwise loss J over the data. A problem in $\mathbb{R}^d$, d large.

Regression: this course, and PINNs

$$\mathcal{L}_{\mathrm{MSE}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \big\|\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i\big\|_2^2$$

MSE = maximum likelihood for Gaussian noise [9]:

$$\boldsymbol{y}_i \mid \boldsymbol{x}_i \sim \mathcal{N}\big(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), s^2 I_m\big) \text{ indep.} \implies -\ln \prod_{i=1}^{N} p_{\boldsymbol{\theta}}(\boldsymbol{y}_i \mid \boldsymbol{x}_i) = \frac{N}{2s^2}\,\mathcal{L}_{\mathrm{MSE}}(\boldsymbol{\theta}) + \frac{Nm}{2}\ln(2\pi s^2)$$

Sometimes: regularization

$$\mathcal{L}_{\mathrm{reg}}(\boldsymbol{\theta}) = \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}) + \frac{\lambda}{2}\,\|\boldsymbol{\theta}\|_2^2$$

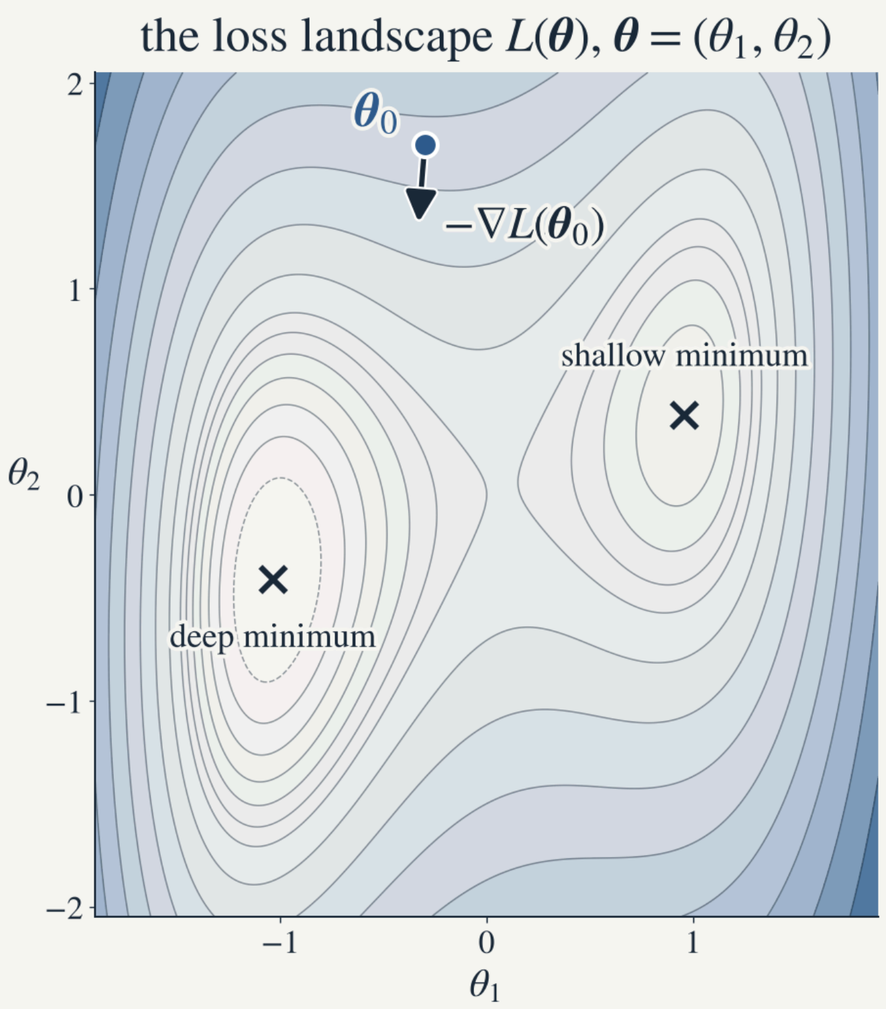
- **Why:** penalizes large weights → smoother $\tilde{f}$, less overfitting when data are few or noisy.
- $\nabla\mathcal{L}_{\mathrm{reg}} = \nabla\mathcal{L}_{\mathcal{D}} + \lambda\boldsymbol{\theta}$: each step shrinks $\boldsymbol{\theta}$ by $(1 - \eta\lambda)$, "weight decay".
- $\lambda > 0$ is a hyperparameter, chosen on validation data.

Classification (C classes): softmax output, cross-entropy

$$\mathcal{L}_{\mathrm{CE}}(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\ln\hat{y}_{i,c}, \qquad \hat{y}_{i,c} = \frac{e^{a_{i,c}}}{\sum_{j=1}^{C} e^{a_{i,j}}} \quad (\text{softmax})$$

[9] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, ch. 5.5 and 7.1.

III. Training is an optimization problem



$$\boldsymbol{\theta}^* = \arg\min \mathrm{L}(\boldsymbol{\theta})$$

d unknowns, L nonlinear: **no formula** for θ*.

1 · Iterative. Start from $\boldsymbol{\theta}_0$ and improve: $\boldsymbol{\theta}_0 \to \boldsymbol{\theta}_1 \to \boldsymbol{\theta}_2 \to \dots$ like the radio knobs.

2 · Gradient-based. We only see the slope where we stand: $-\nabla \mathrm{L}(\boldsymbol{\theta}_k)$ points downhill (a hiker in the fog).

3 · $\boldsymbol{\theta}_0$ must be chosen, and it matters: different starts can end in different valleys (deep or shallow).

One iteration (repeated thousands of times):

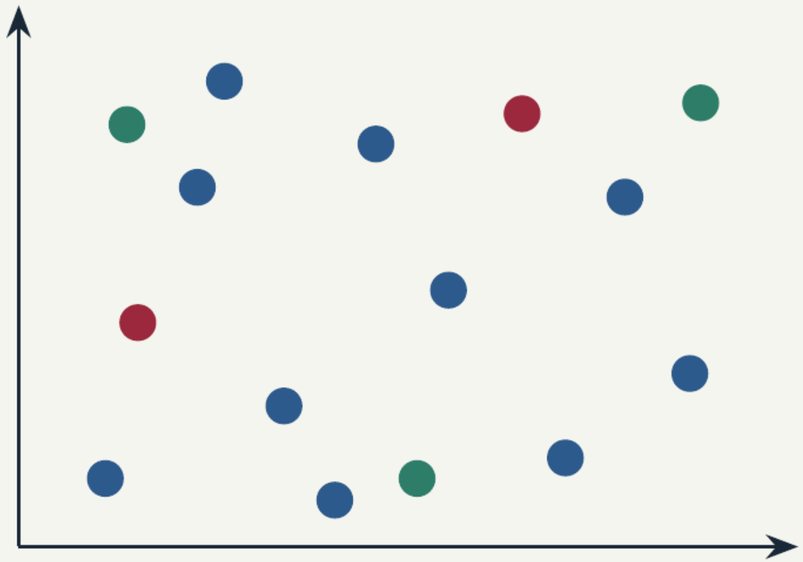
$\boldsymbol{\theta}_k$ → predictions $\tilde{f}(\mathbf{x}_i)$ → loss $\mathrm{L}(\boldsymbol{\theta}_k)$ → gradient $\nabla \mathrm{L}(\boldsymbol{\theta}_k)$ → $\boldsymbol{\theta}_{k+1}$

III. Data: normalize, then split three ways

1 · Normalize inputs and outputs to order 1 (the initial weights and η assume this scale), with statistics of the **training set only**:

$$\tilde{x} = 2\,\frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1 \in [-1, 1] \qquad \text{or} \qquad \tilde{x} = \frac{x - \mu}{s_x}$$

2 · Split at random into three disjoint sets (typically 60 / 20 / 20 %)



- **Training** ($\approx 60\%$): fit $\boldsymbol{\theta} \in \mathbb{R}^d$
  minimize the loss $\mathcal{L}_{\mathcal{D}}$, compute gradients, update θ
- **Validation** ($\approx 20\%$): hyperparameters
  detect overfitting; choose L, p_ℓ, σ, η, batch size, λ
- **Test** ($\approx 20\%$): evaluate $\tilde{f}_{\boldsymbol{\theta}}$
  used once, at the very end

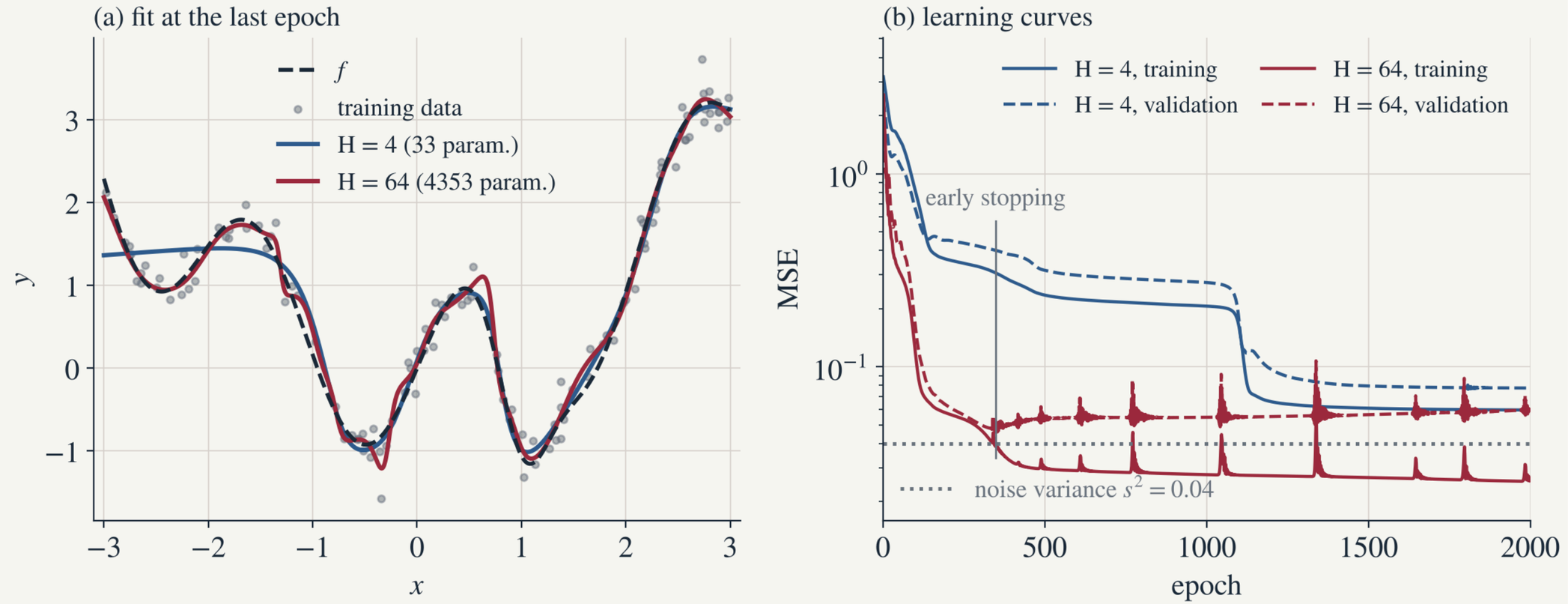
Training: the only data the loss and its gradient see.

Validation: never updates θ; overfitting, hyperparameters.

Test: used **once**, at the end: unbiased error estimate.

III. Too small, too large: under- and overfitting

Example 1 (noise variance $s^2 = 0.04$): MLP $1 \to H \to H \to 1$ (tanh), MSE, Adam $\eta = 10^{-2}$, 2000 epochs, 120 training / 40 validation points.



H = 4: train 0.060, val 0.078, both above s^2: **underfitting**.

H = 64: train ↓ 0.026 while val ↑ 0.046 → 0.061: **overfitting**.

Early stopping: keep θ of the lowest validation loss.

III. Hyperparameters: the validation set decides

1 · CANDIDATES
width H = 4, 8, 16, 32, 64

→

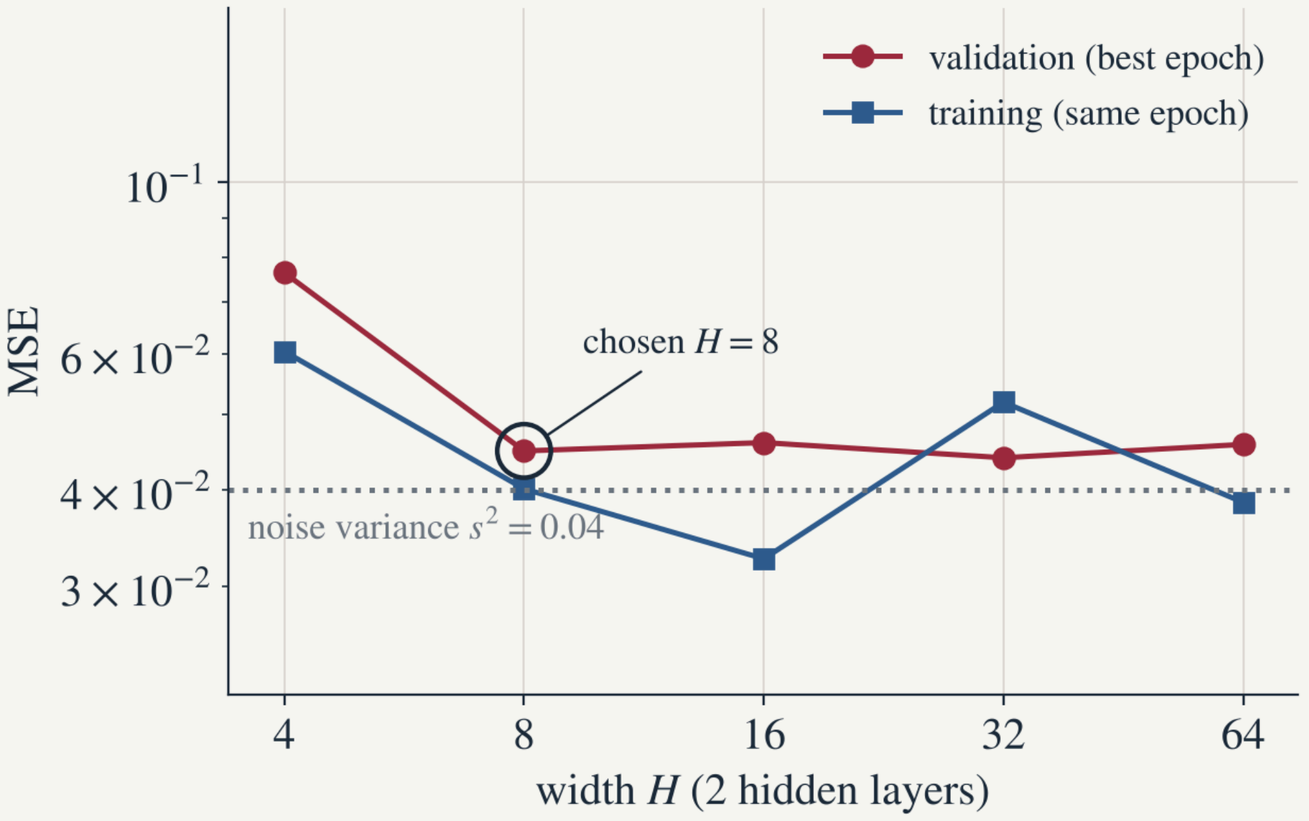
2 · TRAIN EACH
on the training set

→

3 · COMPARE
validation error

→

4 · CHOOSE, THEN TEST
test error, once



Hyperparameter = a choice we make before training: width, depth, σ, η, batch size, epochs…

Not the training error: it always prefers the largest network (blue curve keeps going down).

Example 1: validation (red) stops improving at H = 8 → choose the **smallest good** network, H = 8.

Few data? K-fold cross-validation: repeat with K different validation sets and average.

III. The starting point θ_0

Random, not zero

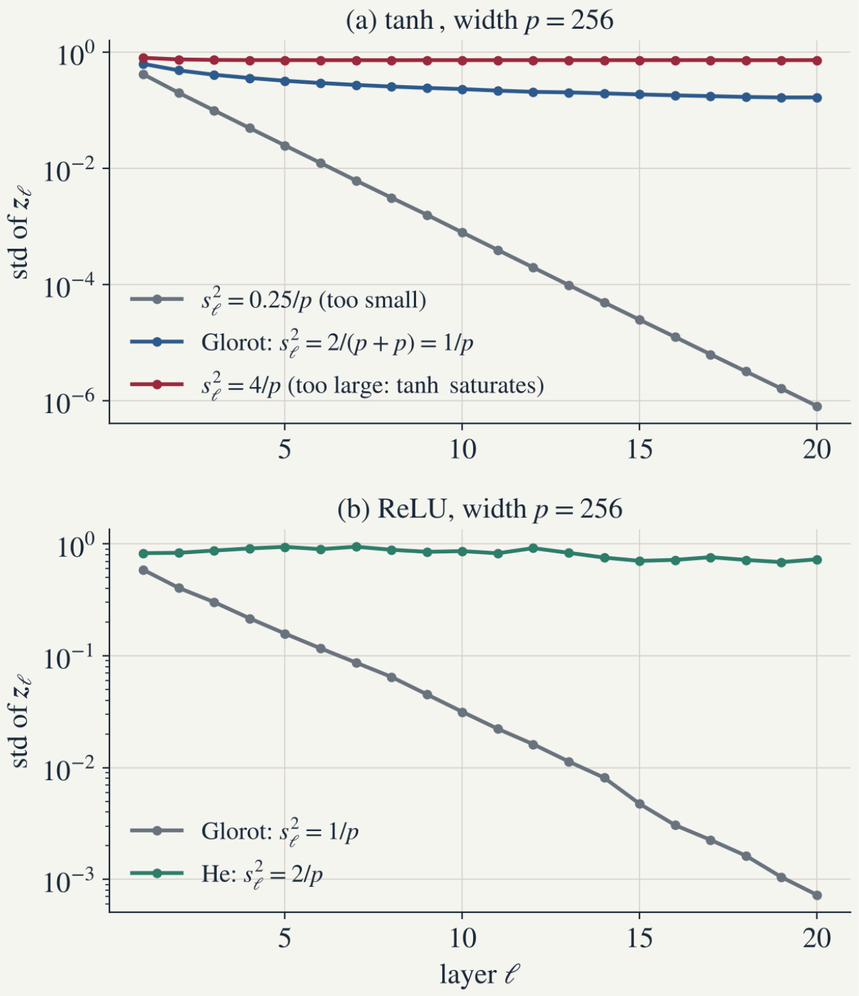
If all weights are equal, all neurons compute the same thing and stay identical. So θ_0 is drawn at random, with small values.

How small? There are "smart" rules

Scale the random weights with the layer size so that signals keep the same order of magnitude through the layers: **Glorot** [10] for tanh, **He** [11] for ReLU.

PyTorch does it for you

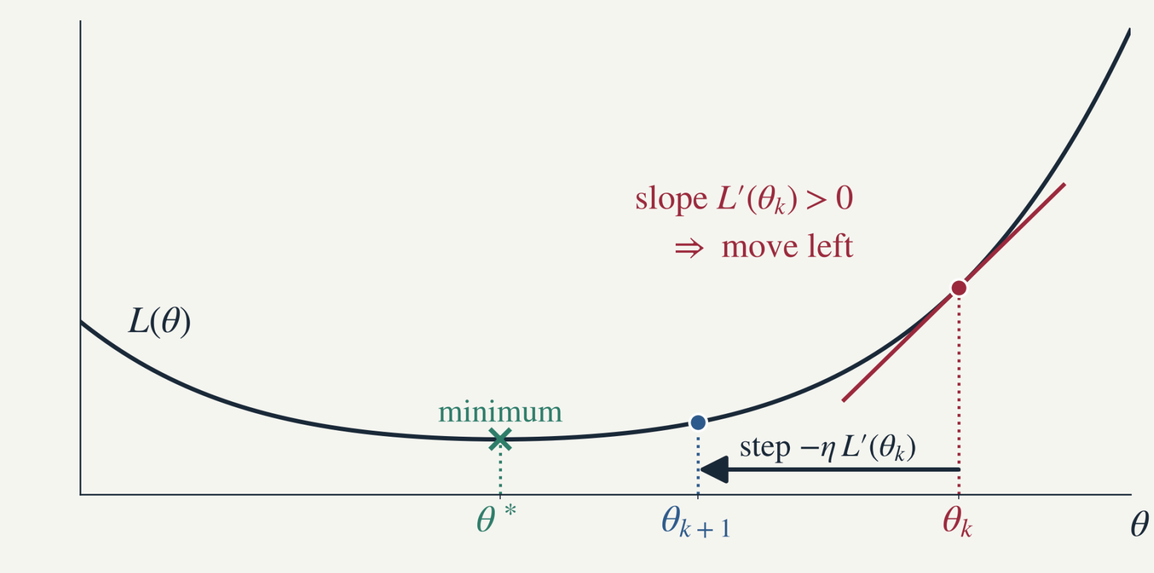
```
layer = nn.Linear(n, p) # already random
nn.init.xavier_normal_(layer.weight) # Glorot
```



[10] X. Glorot, Y. Bengio, AISTATS, 2010. [11] K. He, X. Zhang, S. Ren, J. Sun, ICCV, 2015. Figure: 20 layers of width 256, random inputs.

IV. How to step: use the slope, or also the curvature

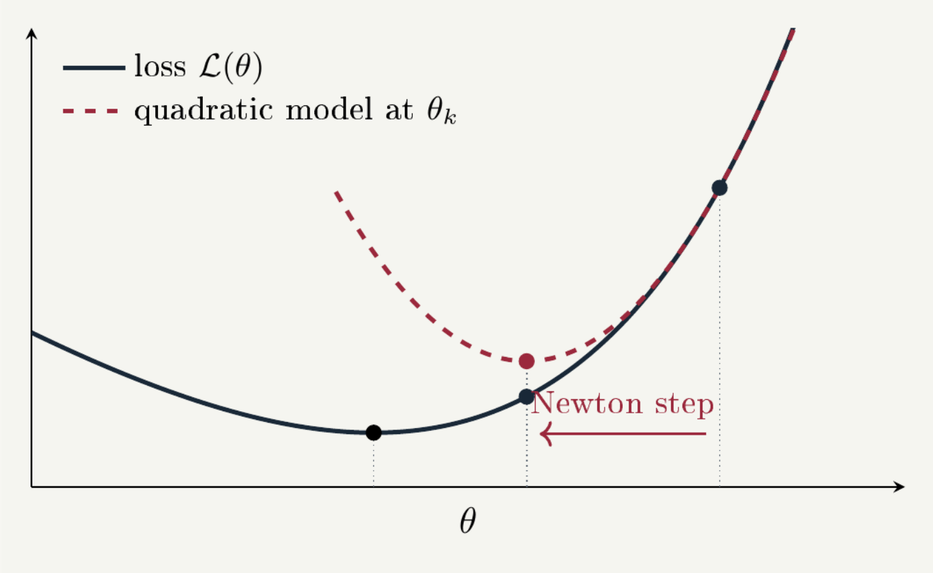
1st order: the slope (gradient descent)



$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\, \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Slope > 0 → go left; steep → big step. Step size η chosen by hand.

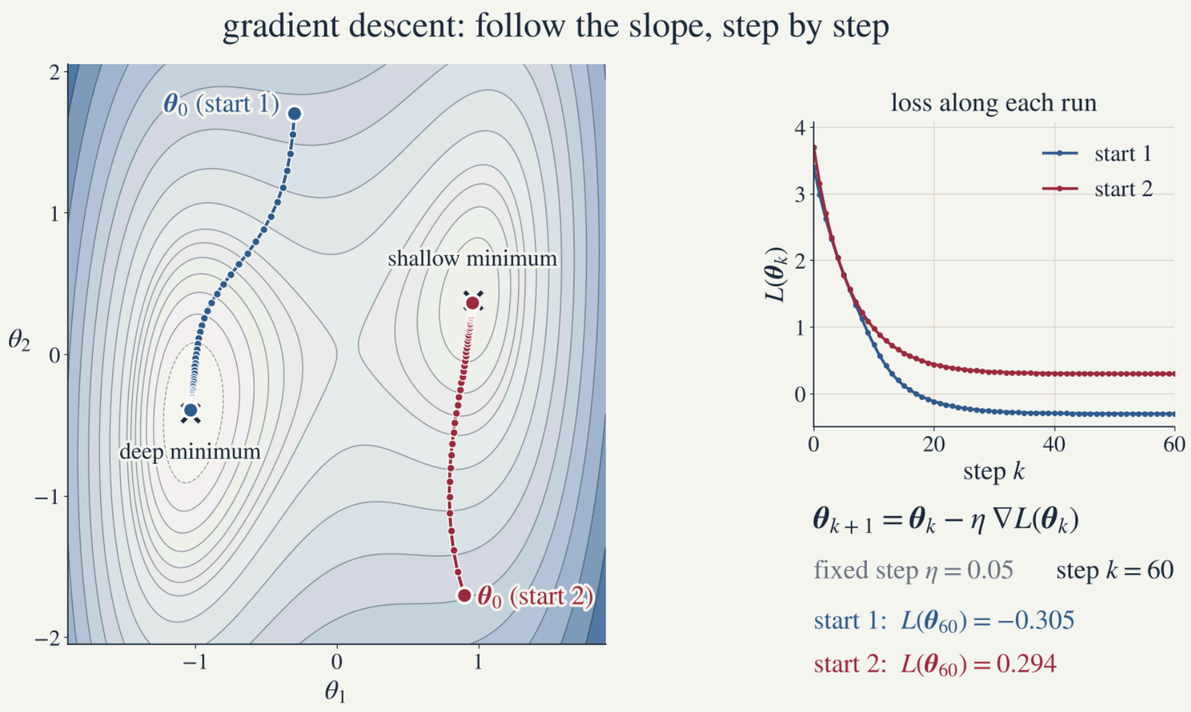
2nd order: also the curvature (Newton)

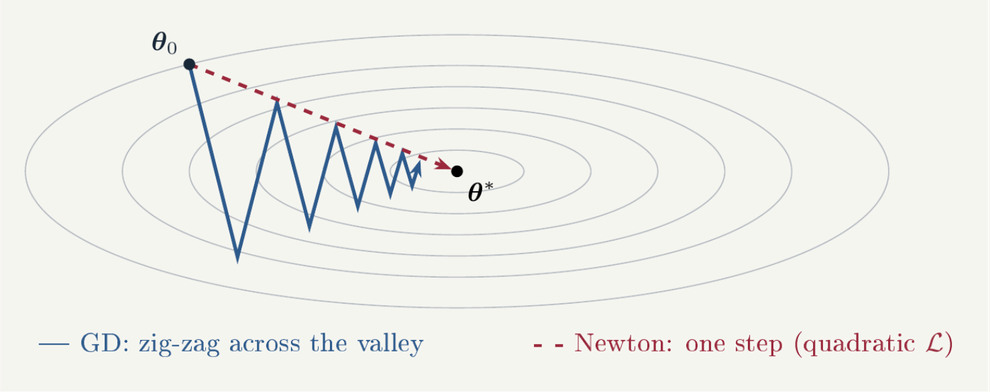


$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \left[\nabla^2 \mathcal{L}(\boldsymbol{\theta}_k)\right]^{-1} \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Fit a parabola (slope + curvature), jump to its bottom. No η, but needs the Hessian (d × d).

IV. In 2D: slope only vs slope and curvature





Narrow valley: GD **zig-zags**; Newton goes to the minimum in **1 step** (quadratic loss).

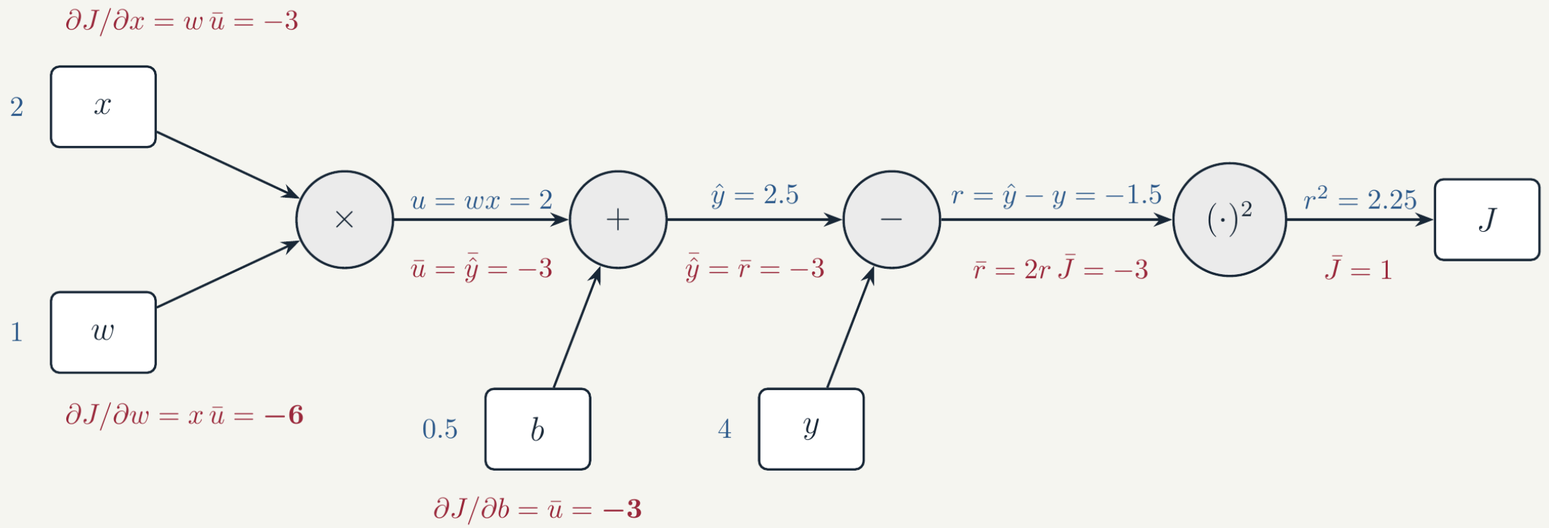
> Both are **local**: neither knows where the deepest valley is.

Gradient descent, η = 0.05, 60 steps, two starts: each ends in its own valley.

Family	Examples	Uses	One step costs	Used for
1st order	GD, SGD, Adam	slope	cheap: O(d)	deep learning; PINNs (start)
2nd order	Newton	slope + curvature	expensive: O(d^3)	small d only
Quasi-Newton	BFGS, L-BFGS	slopes → curvature	moderate: O(rd)	PINNs (d ≈ 10^3–10^4)

IV. The gradient $\nabla\mathcal{L}(\theta)$: automatic differentiation

$J(w,b) = \big(wx + b - y\big)^2$ with $x = 2,\ y = 4,\ w = 1,\ b = 0.5$



Exact, not finite differences (d + 1 evaluations, lost digits).

Reverse mode: all d partial derivatives for a few times the cost of one evaluation [12].

PyTorch: loss.backward() fills p.grad; gradients add up: zero_grad() first.

PINNs: ∂ũ/∂x, ∂²ũ/∂x² by the same AD (autograd.grad, create_graph=True).

For an MLP (backpropagation): $\boldsymbol{a}_\ell = W_\ell\,\boldsymbol{z}_{\ell-1} + \boldsymbol{b}_\ell, \qquad \boldsymbol{z}_\ell = \sigma_\ell(\boldsymbol{a}_\ell)$

$$\boldsymbol{\delta}_L = \frac{\partial J}{\partial \boldsymbol{a}_L}, \qquad \boldsymbol{\delta}_\ell = \big(W_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}\big) \odot \sigma_\ell'(\boldsymbol{a}_\ell), \qquad \frac{\partial J}{\partial W_\ell} = \boldsymbol{\delta}_\ell\,\boldsymbol{z}_{\ell-1}^\top, \qquad \frac{\partial J}{\partial \boldsymbol{b}_\ell} = \boldsymbol{\delta}_\ell$$

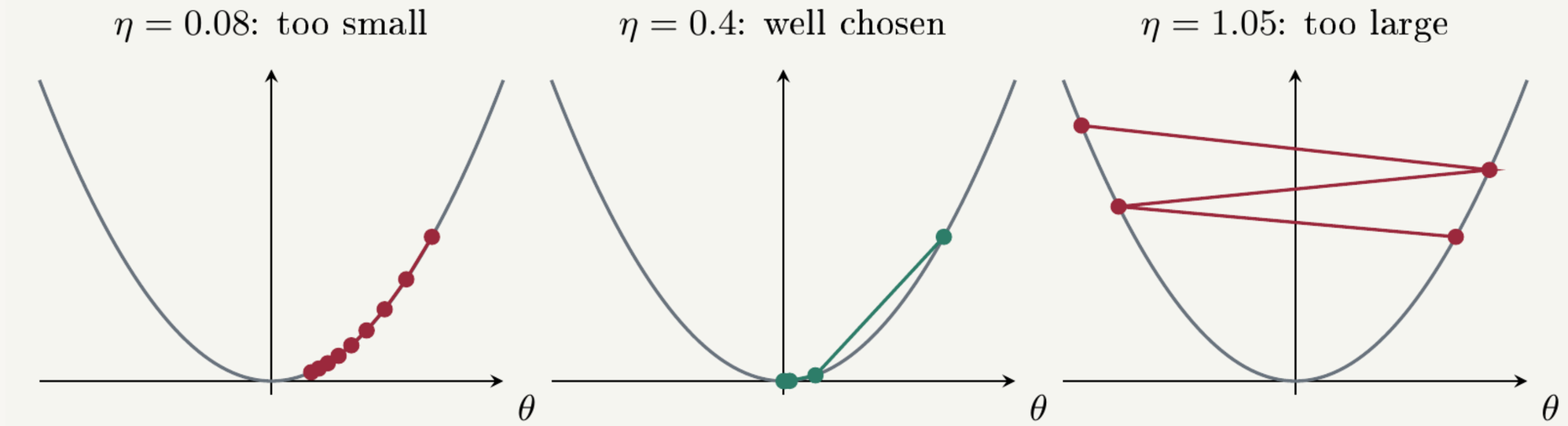
[12] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic differentiation in machine learning: a survey, JMLR, 2018.

IV. Gradient descent: everything depends on η

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

η: learning rate (step size). Model problem:

$$\mathcal{L}(\theta) = \theta^2 : \quad \theta_{k+1} = (1 - 2\eta)\,\theta_k \implies \text{converges} \iff 0 < \eta < 1$$



$$\mathcal{L}(\boldsymbol{\theta}) = \tfrac{1}{2}\,\boldsymbol{\theta}^\top A\,\boldsymbol{\theta},\ A \succ 0 : \quad \text{converges} \iff 0 < \eta < \frac{2}{\lambda_{\max}}, \qquad \|\boldsymbol{\theta}_{k+1}\| \le \frac{\kappa - 1}{\kappa + 1}\|\boldsymbol{\theta}_k\| \text{ for } \eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Two weaknesses: **i)** ill-conditioning ($\kappa \gg 1$) → zig-zag → **Momentum**; **ii)** one η for d parameters of different scales → **RMSProp**; both → **Adam**.

IV. Two fixes of GD: Momentum and RMSProp

Full batch here: $\mathbf{g}_k = \nabla\mathcal{L}(\boldsymbol{\theta}_k)$. Both keep an exponential moving average, with $\beta \in]0, 1[$ (typically 0.9).

Momentum · fixes i) zig-zag

$$\begin{cases} \boldsymbol{v}_{k+1} = \beta\, \boldsymbol{v}_k + \eta\, \boldsymbol{g}_k, & \boldsymbol{v}_0 = \mathbf{0} \\ \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{v}_{k+1} \end{cases}$$

$\mathbf{v}$ averages the past gradients:

- components of constant sign (along the valley) add up: if $\mathbf{g}_k = \mathbf{g}$, then $\mathbf{v}_k \to \eta\mathbf{g} / (1 - \beta)$, a step **10×** larger for $\beta = 0.9$;
- alternating components (across the valley) are damped: amplitude $\to \eta\mathbf{g} / (1 + \beta) \approx 0.53\, \eta\mathbf{g}$, 19× smaller.

RMSProp [13] · fixes ii) one η for all

$$\begin{cases} \boldsymbol{s}_{k+1} = \beta\, \boldsymbol{s}_k + (1 - \beta)\, \boldsymbol{g}_k^2, & \boldsymbol{s}_0 = \mathbf{0} \\ \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\, \dfrac{\boldsymbol{g}_k}{\varepsilon_a + \sqrt{\boldsymbol{s}_{k+1}}} \end{cases}$$

Square, root and division are **component-wise**: each parameter gets its own step.

- if $\mathbf{g}_k = \mathbf{g}$, then $\mathbf{s}_k \to \mathbf{g}^2$, and the step $\to \eta\, \mathrm{sign}(\mathbf{g})$: independent of the scale of the gradient;
- $\varepsilon_a \approx 10^{-8}$ avoids dividing by 0.

Momentum: B. T. Polyak, USSR Comput. Math. Math. Phys., 1964. [13] G. Hinton, N. Srivastava, K. Swersky, Neural networks for machine learning, lecture 6e, 2012.

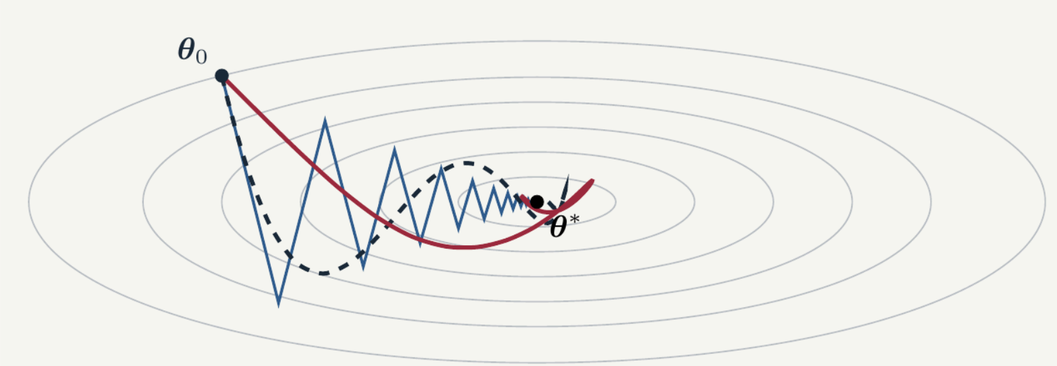
IV. Adam = Momentum + RMSProp + bias correction [14]

$$m_0 = v_0 = 0$$

for $k = 1, \dots, K$:

$g_k = \nabla \mathcal{L}(\theta_{k-1})$	gradient (backpropagation)
$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k$	1st moment: momentum
$v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2$	2nd moment: RMSProp
$\hat{m}_k = \frac{m_k}{1-\beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1-\beta_2^k}$	bias correction
$\theta_k = \theta_{k-1} - \alpha \frac{\hat{m}_k}{\varepsilon + \sqrt{\hat{v}_k}}$	component-wise

$\alpha = 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \varepsilon = 10^{-8}$



$\mathcal{L}(\theta) = (\theta_1^2 + 10\,\theta_2^2)/2$, same start for the three methods.

Bias correction: $m_1 = 0.1\, g_1$ is biased towards 0; $\hat{m}_1 = g_1$.

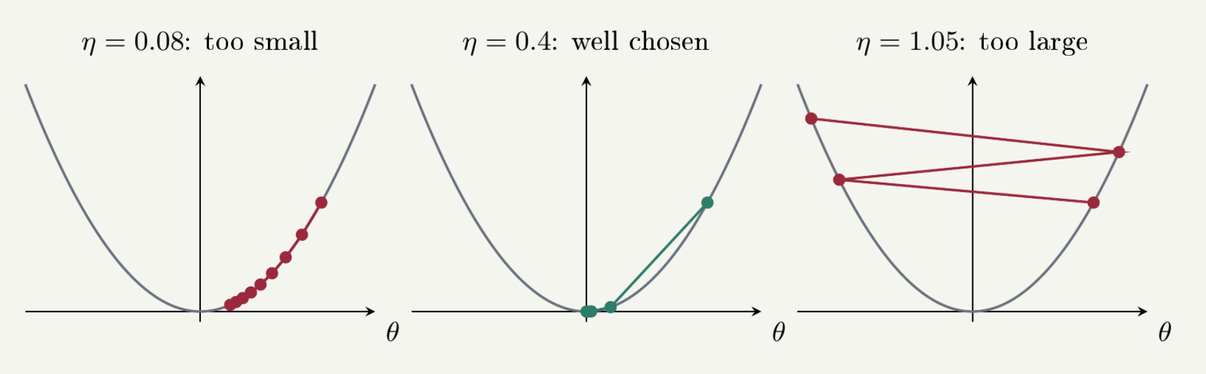
First step $\approx \alpha\, \mathrm{sign}(g_1)$: every parameter moves by α.

Default optimizer; first stage of PINN training. α = learning rate (lr).

[14] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, ICLR, 2015. In PyTorch: torch.optim.Adam(model.parameters(), lr=1e-3).

IV. The learning rate η: not too small, not too large

On a parabola ($\theta \mapsto \theta^2$, previous slide):

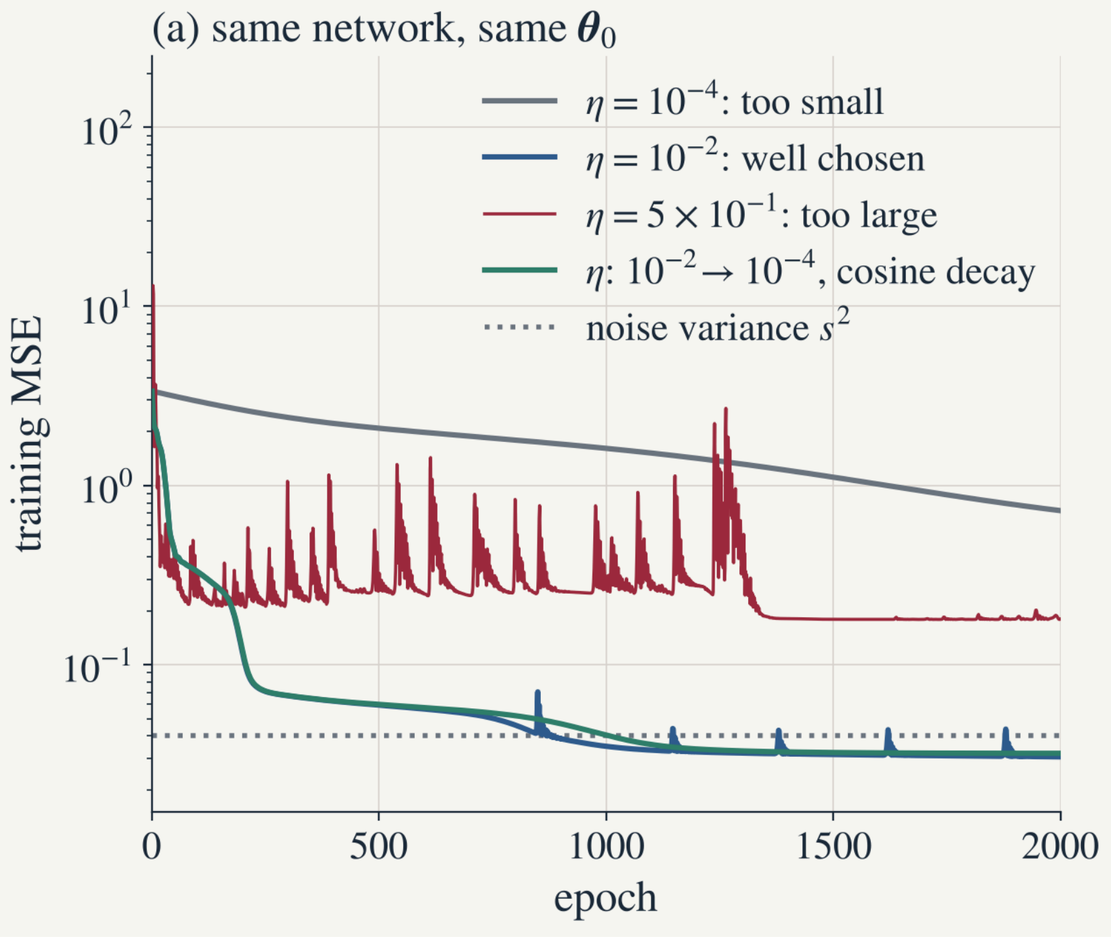


Too small: tiny steps, the loss barely moves.

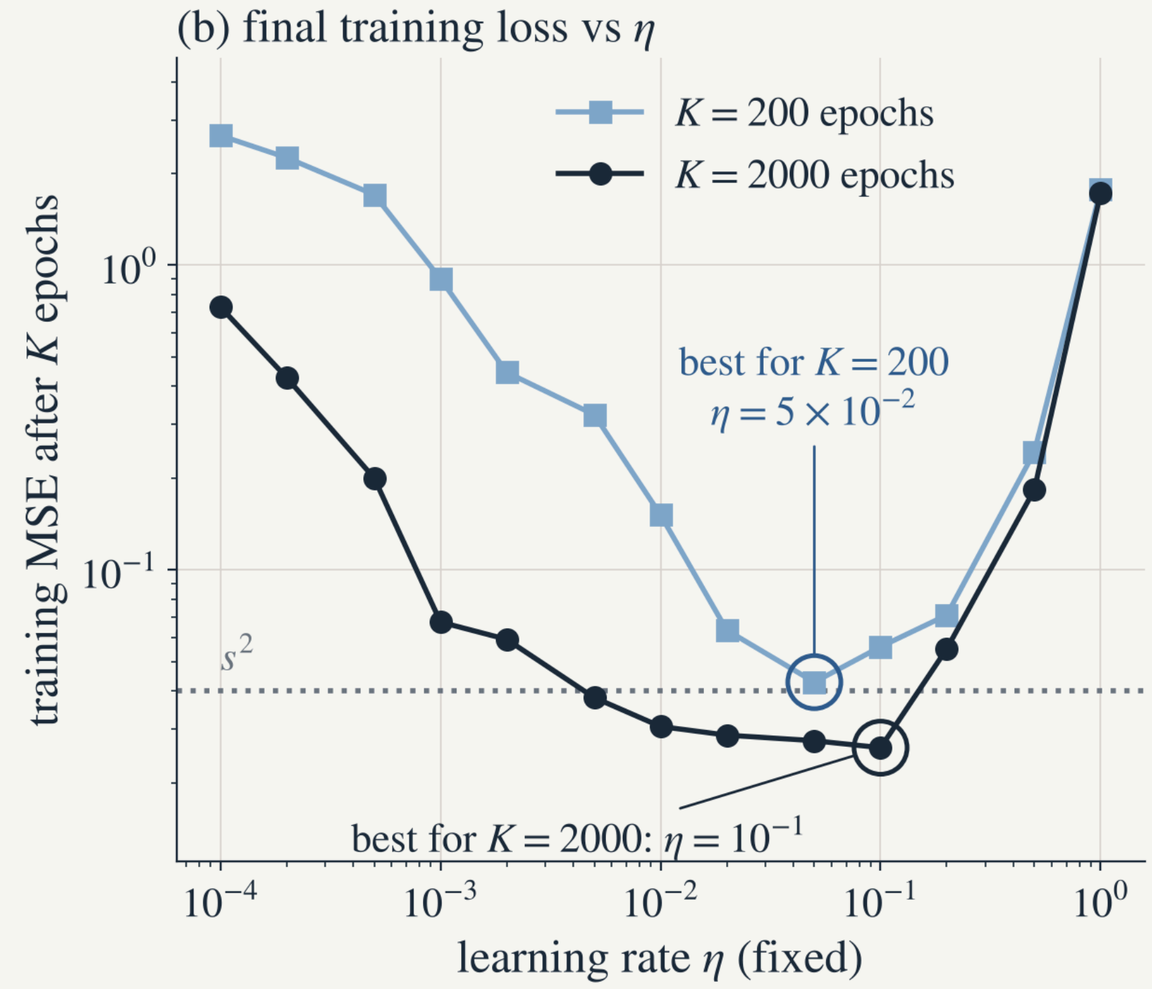
Well chosen: fast decrease, down to the noise level.

Too large: steps jump over the valley: oscillations, or divergence.

Right: the same story on a real network (example 1, Adam, 2000 epochs, same θ_0).



IV. The right η depends on how long you train



How to read it: each dot is a whole training with one fixed η; we plot the error reached after K epochs.

Short training (K = 200): only large steps go far enough. Small η: not enough time.

Long training (K = 2000): small η catch up; a wide range works ($5 \cdot 10^{-3}$ to 10^{-1}).

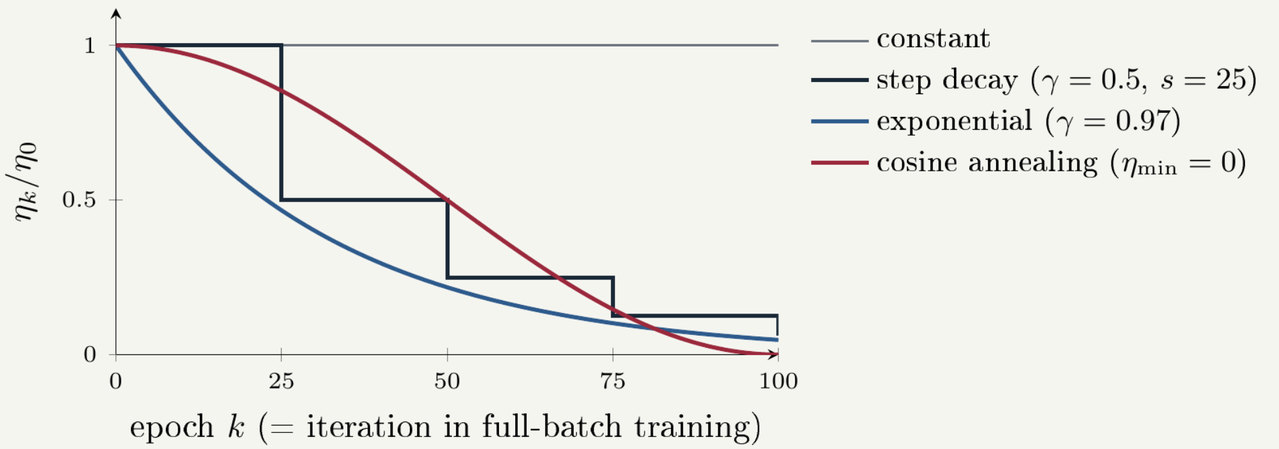
Too large ($\eta \geq 0.5$): fails whatever the budget.

In practice: start with Adam, $\eta = 10^{-3}$; try ×10 and ÷10; look at the loss curve; choose η together with the number of epochs.

IV. Learning-rate schedules: large steps, then small steps

With mini-batch noise or with Adam (steps ≈ α even near the minimum), a fixed η keeps oscillating. Decrease it with the **epoch** k:

step decay	$\eta_k = \eta_0 \, \gamma^{\lfloor k/s \rfloor}$
exponential	$\eta_k = \eta_0 \, \gamma^{k}$
cosine	$\eta_k = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\big(1 + \cos(\pi k / K)\big)$
on plateau	$\eta \leftarrow \gamma \, \eta$ if $\mathcal{L}$ has not decreased for P epochs

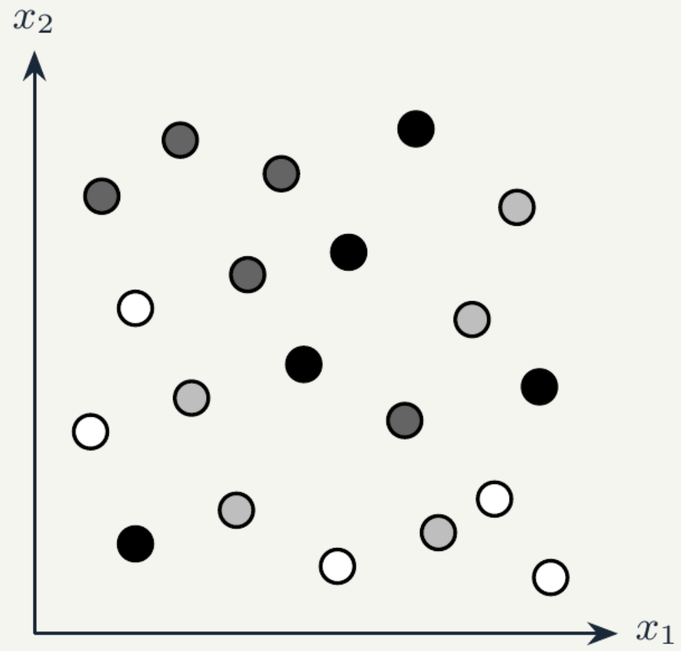


Schedule	PyTorch (torch.optim.lr_scheduler)	Call
step decay	StepLR(opt, step_size=s, gamma=γ)	scheduler.step()
cosine	CosineAnnealingLR(opt, T_max=K)	scheduler.step()
on plateau	ReduceLROnPlateau(opt, factor=0.5, patience=20)	scheduler.step(loss)

Once per epoch, after the optimizer steps. Often a short **warm-up** first (η from 0 to η_0 linearly).

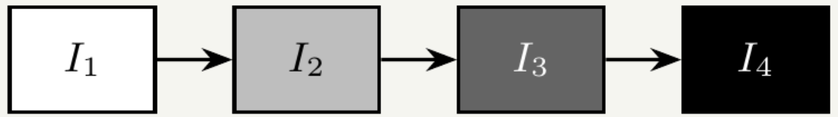
Cosine annealing: I. Loshchilov, F. Hutter, SGDR: stochastic gradient descent with warm restarts, ICLR, 2017.

IV. Mini-batches: update θ after each small packet of data



training set $(N = 20)$, shuffled into $M = 4$ mini-batches of $B = 5$

one epoch $= M$ updates of $\boldsymbol{\theta}$:



update 1 update 2 update 3 update 4

update j: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}_{I_j}(\boldsymbol{\theta})$

Update with batch j (any optimizer: GD, Adam…):

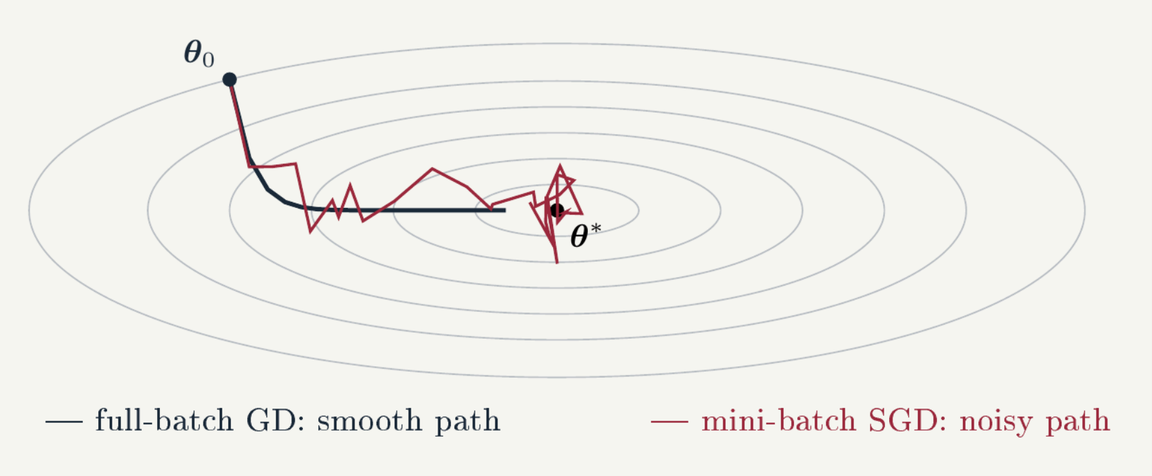
$$\mathcal{L}_{I_j}(\boldsymbol{\theta}) = \frac{1}{|I_j|} \sum_{i \in I_j} J\big(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i\big), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}_{I_j}(\boldsymbol{\theta}), \qquad j = 1, \ldots, M = \lceil N/B \rceil$$

1 epoch = one pass over the data = **M = N / B updates** (instead of 1).

B = N: full batch · B = 1: “SGD”.

IV. Mini-batches: noisier steps, but faster per epoch

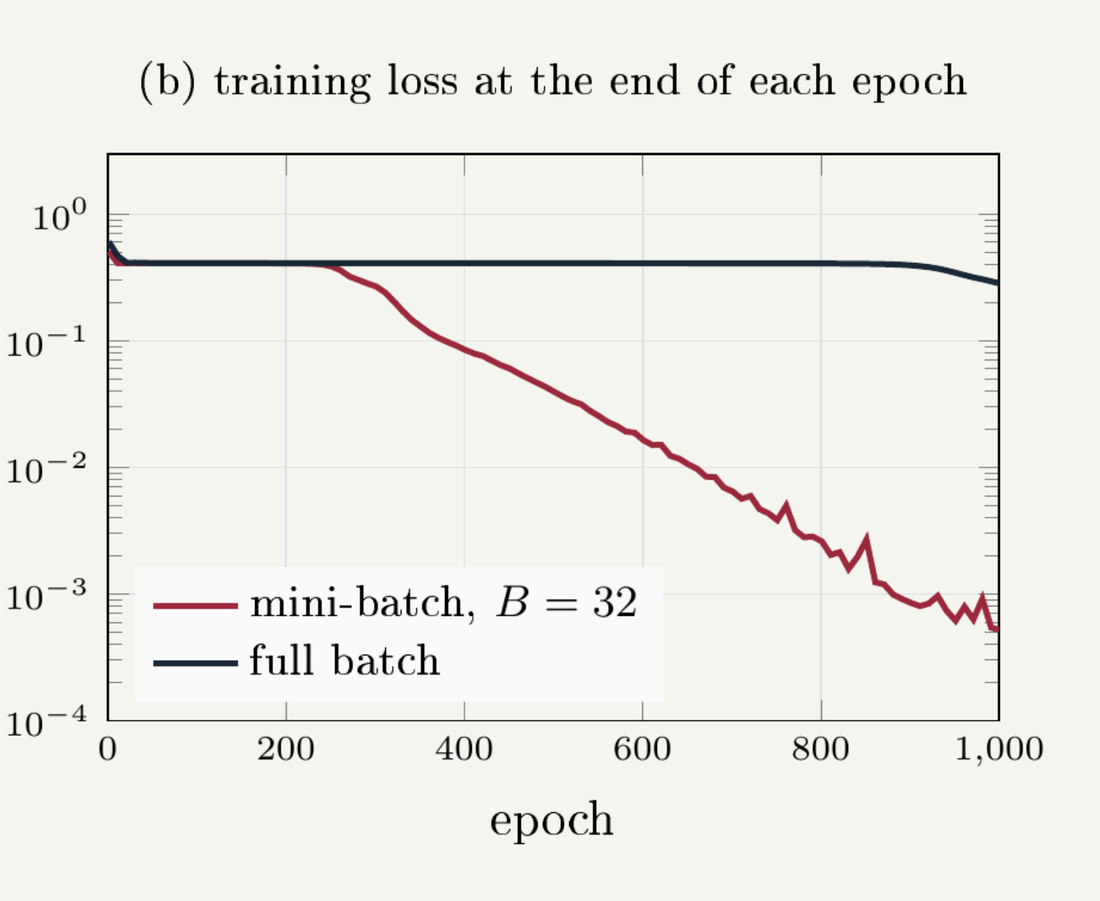
1 · Each step is noisy



A batch sees only part of the data: its gradient is right **on average**, but each step zig-zags around the full-batch path.

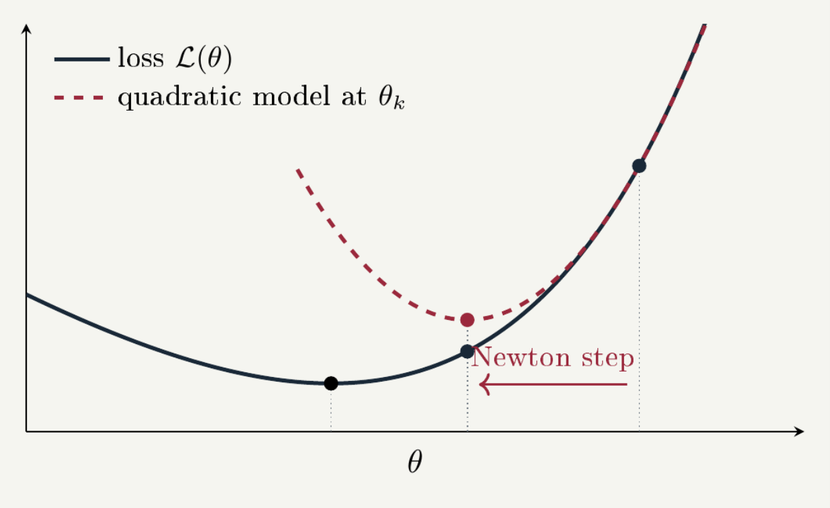
Setting of the right plot: N = 256 samples, Adam, same θ_0, 1000 epochs. Full batch: 1000 updates. B = 32: 8 per epoch, 8000 updates.

2 · But many more steps for the same cost



IV. Second order: Newton, Gauss–Newton, L-BFGS

Newton: minimize the quadratic model



$$\theta_{k+1} = \theta_k - \frac{\mathcal{L}'(\theta_k)}{\mathcal{L}''(\theta_k)}$$

Quadratic convergence near a minimum with $\nabla^2\mathcal{L} > 0$. Cost: $O(d^2)$ memory, $O(d^3)$ solve.

Gauss–Newton / Levenberg–Marquardt (MSE: $\mathcal{L} = \|r\|^2/N$)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \left(\mathsf{J}^\top \mathsf{J} + \mu_k I\right)^{-1} \mathsf{J}^\top \boldsymbol{r}, \qquad \boldsymbol{r} = \left(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i\right)_i, \quad \mathsf{J} = \frac{\partial \boldsymbol{r}}{\partial \boldsymbol{\theta}}$$

Hessian ≈ (2/N) $\mathsf{J}^\top\mathsf{J}$: first derivatives only, positive semi-definite. $\mu = 0$: Gauss–Newton.

Quasi-Newton: $H_k \approx [\nabla^2\mathcal{L}]^{-1}$ from gradients only (BFGS)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k H_k \nabla\mathcal{L}(\boldsymbol{\theta}_k), \qquad H_{k+1}\boldsymbol{q}_k = \boldsymbol{s}_k, \quad \boldsymbol{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k, \quad \boldsymbol{q}_k = \nabla\mathcal{L}(\boldsymbol{\theta}_{k+1}) - \nabla\mathcal{L}(\boldsymbol{\theta}_k)$$

$$H_{k+1} = \left(I - \rho_k \boldsymbol{s}_k \boldsymbol{q}_k^\top\right) H_k \left(I - \rho_k \boldsymbol{q}_k \boldsymbol{s}_k^\top\right) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^\top, \qquad \rho_k = \frac{1}{\boldsymbol{q}_k^\top \boldsymbol{s}_k}$$

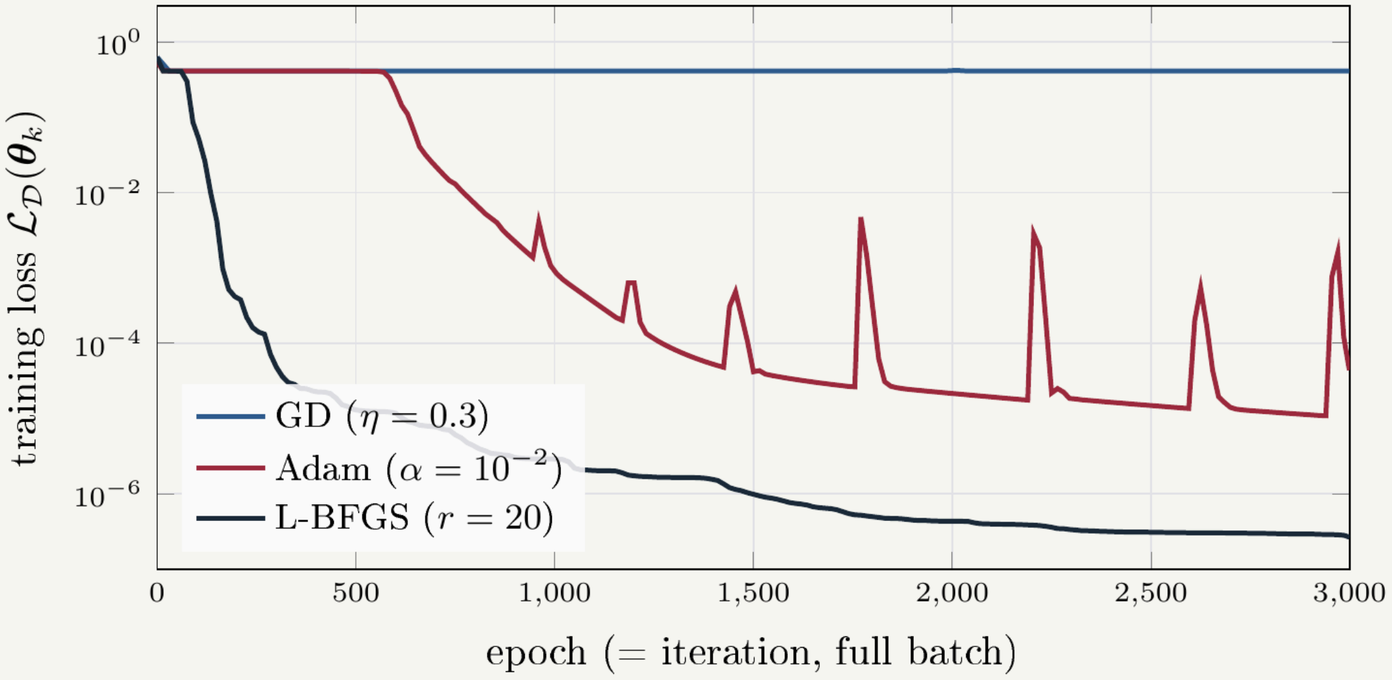
L-BFGS [15]: never forms H_k, keeps the last r = 10–50 pairs (**s**, **q**): $O(rd)$ memory.

No learning rate to tune: η_k by a line search along $\mathbf{p}_k = -H_k\nabla\mathcal{L}(\boldsymbol{\theta}_k)$, Wolfe conditions [16], $0 < c_1 < c_2 < 1$; they ensure $\mathbf{q}_k^\top\mathbf{s}_k > 0$, so H_{k+1} stays positive definite.

$$\mathcal{L}(\boldsymbol{\theta}_k + \eta_k \boldsymbol{p}_k) \le \mathcal{L}(\boldsymbol{\theta}_k) + c_1\, \eta_k\, \nabla\mathcal{L}(\boldsymbol{\theta}_k)^\top \boldsymbol{p}_k, \qquad \nabla\mathcal{L}(\boldsymbol{\theta}_k + \eta_k \boldsymbol{p}_k)^\top \boldsymbol{p}_k \ge c_2\, \nabla\mathcal{L}(\boldsymbol{\theta}_k)^\top \boldsymbol{p}_k$$

[15] D. C. Liu, J. Nocedal, Math. Programming, 1989. [16] J. Nocedal, S. J. Wright, Numerical Optimization, Springer, 2006 ($c_1 = 10^{-4}$, $c_2 = 0.9$; PyTorch: strong Wolfe).

IV. GD, Adam, L-BFGS on the same problem



sin(2πx), N = 256, d = 481, full batch, same θ_0, 3000 iterations.

GD (η = 0.3; larger diverges): stuck, $\mathcal{L} \approx 0.41$.

Adam (α = 10^{-2}): leaves the plateau near epoch 630, $\mathcal{L} \approx 10^{-5}$–$10^{-4}$, spikes.

L-BFGS (r = 20): leaves it near iteration 90, $\mathcal{L} \approx 3 \cdot 10^{-7}$, 3417 evaluations.

⇒ for PINNs: Adam first, then L-BFGS.

Self-scaled BFGS (Oren–Luenberger, 1974)

$$H_k \leftarrow \tau_k H_k, \quad \tau_k = \frac{\boldsymbol{s}_k^\top \boldsymbol{q}_k}{\boldsymbol{q}_k^\top H_k \, \boldsymbol{q}_k}$$

losses orders of magnitude below Adam and L-BFGS, reported on PINN benchmarks [17]

Natural gradient [18–20]

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \, G(\boldsymbol{\theta}_k)^\dagger \nabla \mathcal{L}(\boldsymbol{\theta}_k), \quad G_{ij} = \left\langle \partial_{\theta_i} \tilde{f}_{\boldsymbol{\theta}}, \partial_{\theta_j} \tilde{f}_{\boldsymbol{\theta}} \right\rangle_{\mathcal{H}}$$

$\langle \cdot, \cdot \rangle_{\mathcal{H}}$ induced by the loss (for a PINN: through the PDE operator)

[17] Kiyani et al., CMAME, 2025. [18] Müller, Zeinhofer, ICML 2023. [19] Jnini, Vella, Zeinhofer, arXiv:2402.10680, 2024. [20] Jnini, Vella, TMLR 2025.

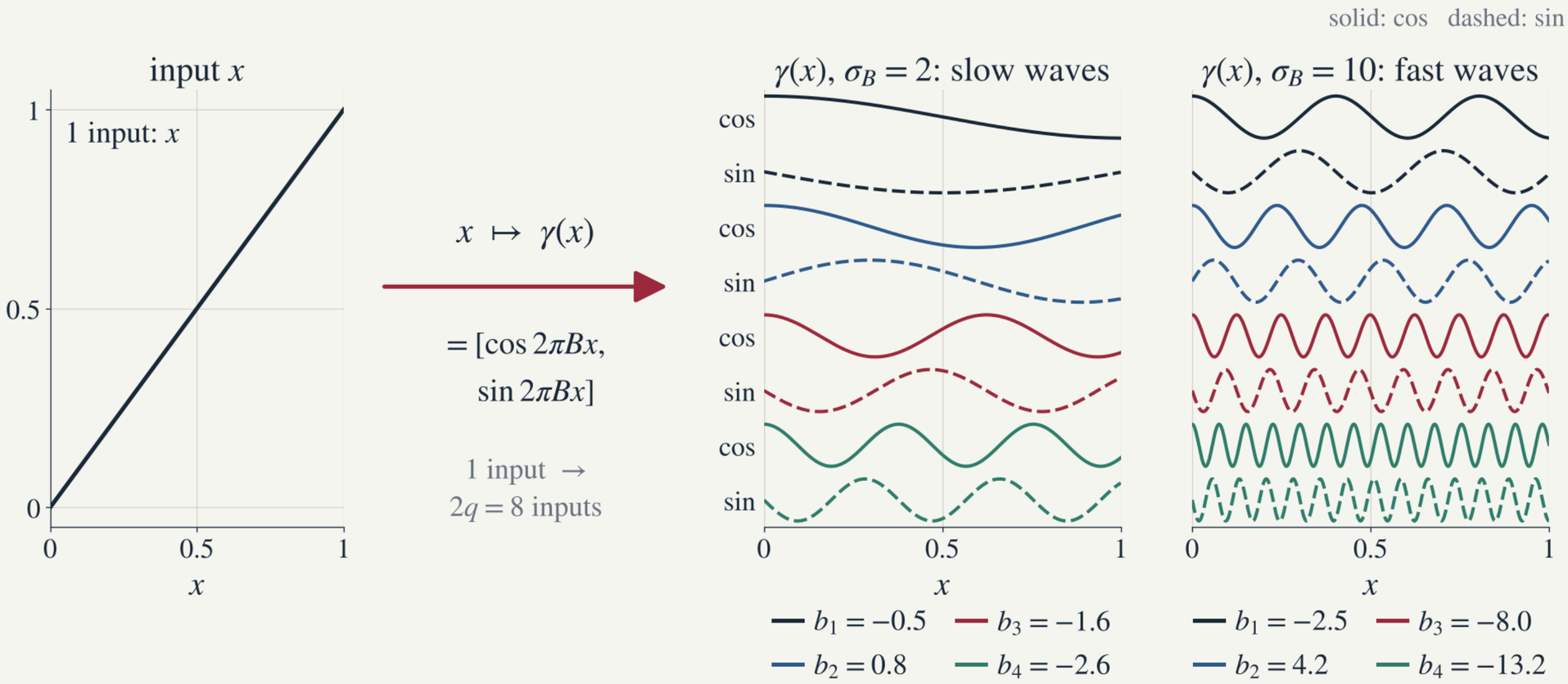
V. Everything in one loop (PyTorch)

```
model = MLP([2, 128, 128, 128, 1])
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = lr_scheduler.CosineAnnealingLR(optimizer, n_epochs)
loss_fn, best = nn.MSELoss(), float("inf")
for epoch in range(n_epochs):
    model.train()
    perm = torch.randperm(n_train) # new shuffle
    for i in range(0, n_train, batch_size):
        idx = perm[i:i + batch_size]
        optimizer.zero_grad() # 1. clear old gradients
        loss = loss_fn(model(X_train[idx]), Y_train[idx]) # 2.
        loss.backward() # 3. gradient (autodiff)
        optimizer.step() # 4. update θ
    scheduler.step() # once per epoch
    model.eval()
    with torch.no_grad(): # no graph needed
        val_loss = loss_fn(model(X_val), Y_val).item()
    if val_loss < best: # early stopping
        best, state = val_loss, copy.deepcopy(model.state_dict())
model.load_state_dict(state) # restore the best θ
```

```
# then refine: L-BFGS, full batch
opt = torch.optim.LBFGS(
  model.parameters(),
  max_iter=1, history_size=50,
  line_search_fn="strong_wolfe")
def closure():
  opt.zero_grad()
  loss = loss_fn(model(X_train), Y_train)
  loss.backward()
  return loss
for it in range(n_iter):
  opt.step(closure)
```

The line search re-evaluates $\mathcal{L}$ several times per step: hence the **closure**. max_iter=1: one L-BFGS iteration per step() (default 20).

V. Fourier features: feed the network with waves

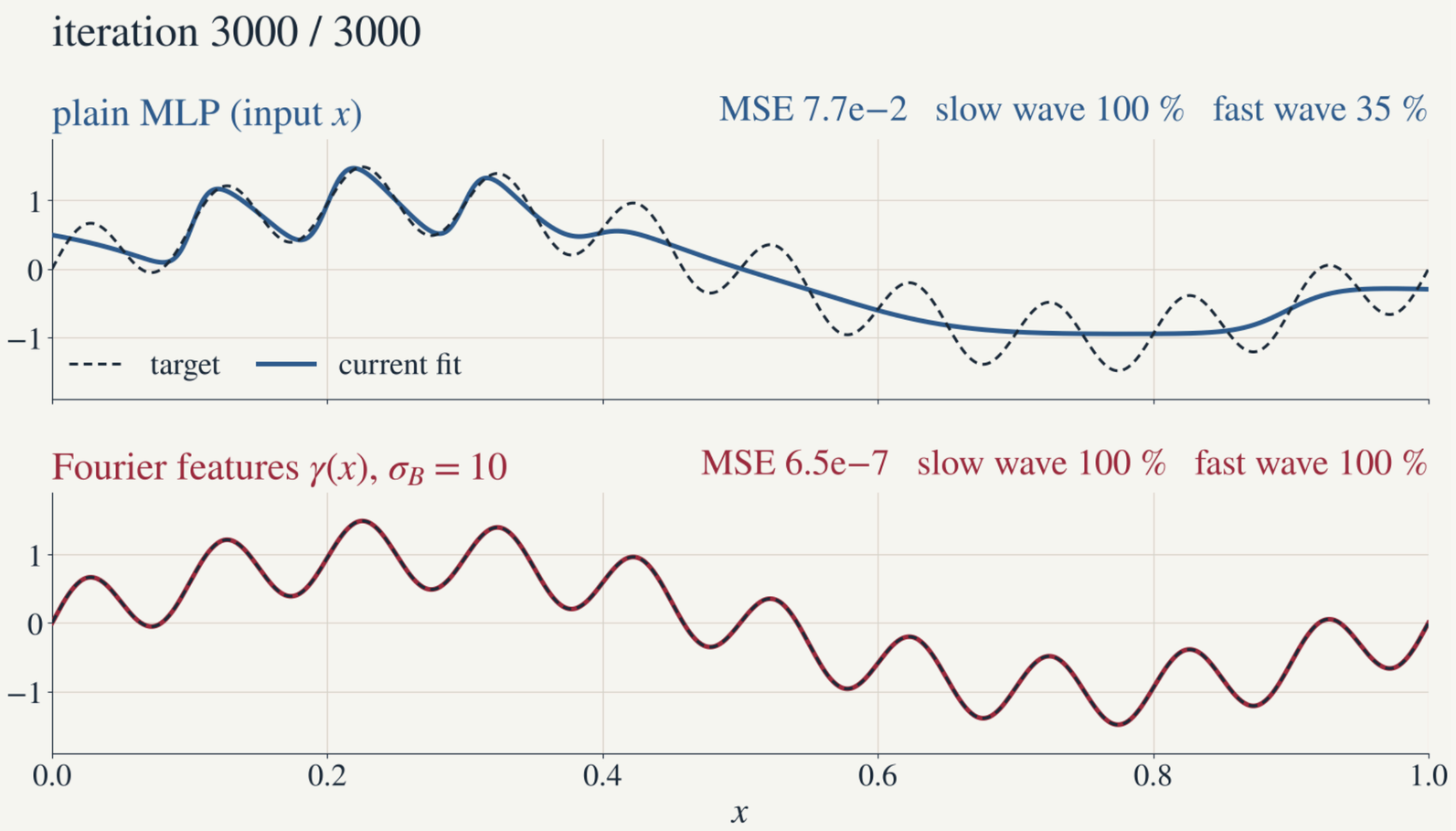


$$\gamma(\boldsymbol{v}) = \big[\cos(2\pi B\boldsymbol{v}),\ \sin(2\pi B\boldsymbol{v})\big] \in \mathbb{R}^{2q}, \qquad B \in \mathbb{R}^{q\times 2}, \quad B_{ij} \sim \mathcal{N}(0, \sigma_B^2)$$

B: random frequencies, drawn once and **fixed**. σ_B = how fast the waves are: a hyperparameter.

[22] M. Tancik et al., Fourier features let networks learn high frequency functions in low dimensional domains, NeurIPS, 2020.

V. Effect in 1D: the fast wave is learned at once



Target: $\sin 2\pi x + 0.5 \sin(2\pi \cdot 10x)$, 200 points, Adam 10^{-3}, 3000 iterations.

Plain MLP: slow wave after 450 iterations; fast wave only **35 %** at 3000 (learned around 5000). MSE 0.077.

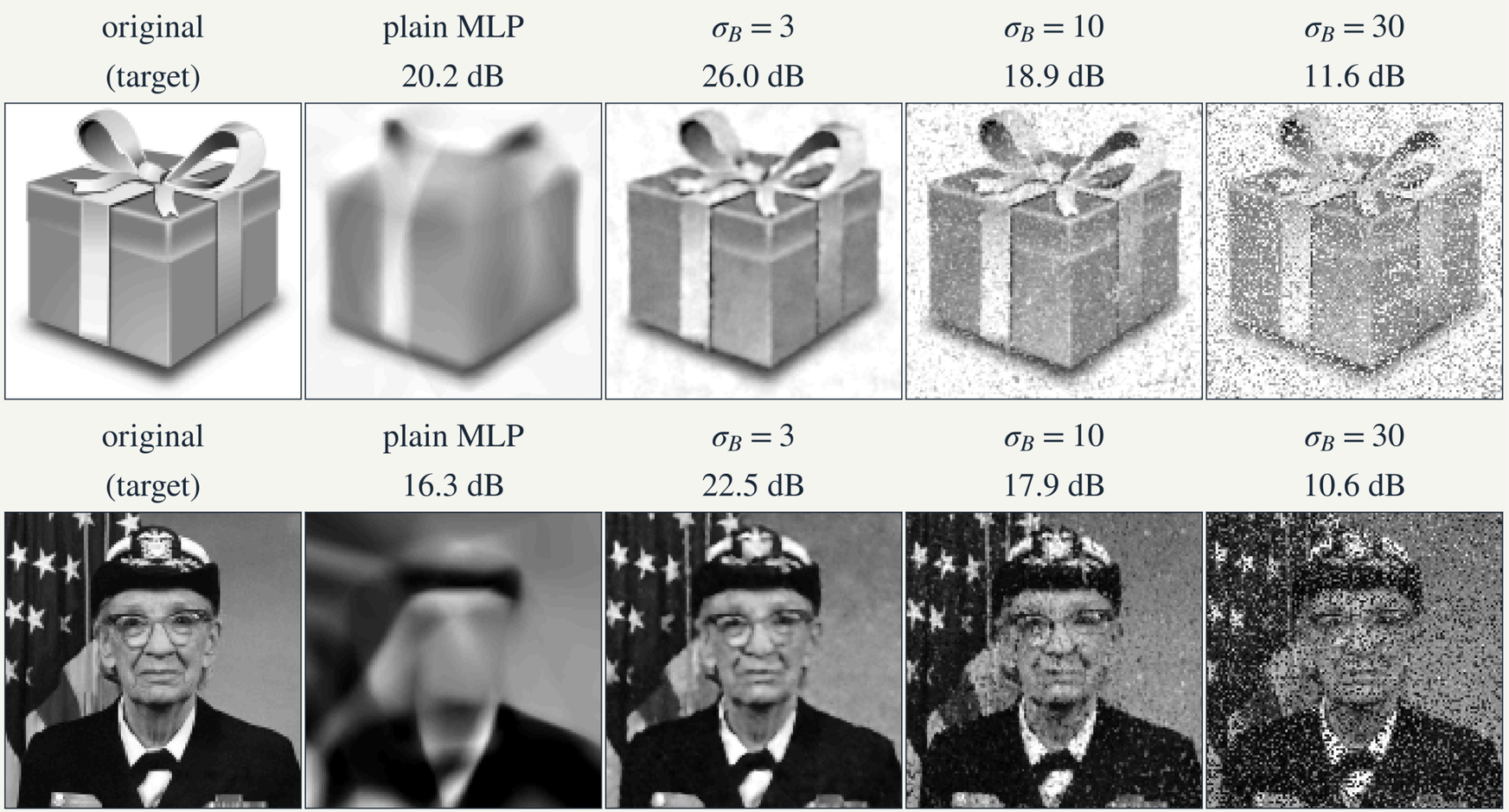
Fourier features (σ_B = 10): both waves within **50** iterations. MSE $6.5 \cdot 10^{-7}$.

Networks learn **low frequencies first**: the “spectral bias” [21].

[21] N. Rahaman et al., On the spectral bias of neural networks, ICML, 2019.

V. On images: Fourier features sharpen the picture

Input (x, y) → grey level. Plain MLP vs Fourier features γ(x, y) with three scales σ_B, same network, same 30 epochs.



q = 128 → [256, 128, 128, 128, 1] ReLU, Adam 10^{-3}, batches of 64, 30 epochs. dB: validation PSNR = 10 $\log_{10}(1/\text{MSE})$.

Matrix **B fixed**, not trained; σ_B = frequency scale, a hyperparameter.

σ_B = 10, 30: noise between training pixels: overfitting.

σ_B = 3 chosen on validation; test PSNR **26.0 dB** and **22.7 dB**.

[21] N. Rahaman et al., ICML, 2019. [22] M. Tancik et al., NeurIPS, 2020.

NEXT: HANDS-ON

Now you train your own networks, in a notebook

We practice everything seen today with PyTorch, step by step. The same code will be the starting point of our PINNs.

